

VPS

Índice

1 Despliegue en VPS con Docker.....	1
1.1 Para quién es esta guía	5
1.2 Qué vais a conseguir al terminar	5
1.3 Cómo seguir la guía.....	5
2 Introducción y arquitectura general	6
2.1 El problema que queremos resolver.....	6
2.2 La arquitectura final.....	6
2.3 Por qué Docker	7
2.4 El flujo de trabajo que vamos a aprender.....	7
2.5 Lo que necesitas antes de empezar.....	7
3 Conceptos previos	9
3.1 ¿Qué es un VPS?	9
3.2 ¿Qué es un dominio y cómo funcionan los DNS?	9
3.3 ¿Qué es SSH?	11
3.4 ¿Qué es Docker?	12
3.5 ¿Qué es un reverse proxy y por qué lo necesitamos?	12
3.6 ¿Qué es Let's Encrypt y por qué necesito HTTPS?	13
4 Conexión al servidor y claves SSH.....	15
4.1 Obtener los datos de acceso	15
4.2 Primera conexión.....	15
4.3 Generar una clave SSH en tu ordenador	16
4.4 Copiar la clave pública al servidor.....	17
4.5 Configurar un alias en	18
4.6 (Opcional) Configurar SSH en el servidor para usar GitHub.....	19
5 Instalación de Docker en el servidor	21
5.1 Actualizar el sistema	21
5.2 Instalar Docker (paso a paso).....	21
5.3 Verificar que Docker funciona	23
5.4 Permitir ejecutar Docker sin	23
5.5 Limpiar las pruebas	24
5.6 Comandos básicos de Docker que vas a usar todo el rato	24
6 Configurar nginx-proxy y HTTPS automático.....	26
6.1 Asegurar que los puertos 80 y 443 están abiertos.....	26

6.2 Crear la red de Docker para el proxy.....	27
6.3 Crear el del proxy	27
6.4 Crear la carpeta	30
6.5 Levantar el proxy.....	30
6.6 Comprobar que el proxy responde	31
6.7 Ver los logs del proxy	31
6.8 Configurar tu dominio en el DNS	31
6.9 Personalizar la configuración de nginx (timeouts, tamaño de subida y más).....	32
7 Desplegar un frontend (React / Vite)	35
7.1 Compilar la app en local.....	35
7.2 Estructura de carpetas en el servidor.....	36
7.3 Subir los archivos compilados al servidor	36
7.4 Crear el	37
7.5 Levantar el contenedor.....	39
7.6 Si tu app usa React Router (rutas del lado cliente)	40
7.7 Actualizar el frontend	41
7.8 Resumen visual del flujo	41
8 Desplegar un backend con base de datos	43
8.1 Asumimos esta estructura de proyecto	43
8.2 Clonar el repo en el servidor	43
8.3 Configurar variables de entorno	44
8.4 Crear el	44
8.5 ¿Cómo se comunican backend y BBDD?	48
8.6 Levantar todo	48
8.7 Migraciones, seeders, cosas a ejecutar dentro del contenedor	49
8.8 Acceder a la base de datos desde fuera (debug, copias, etc.).....	50
8.9 Hacer backups (y restaurarlos) de la base de datos.....	50
8.10 Actualizar el backend	55
8.11 Anexo: Dockerfile básico para Node.js	56
9 Troubleshooting: qué hacer cuando algo falla	57
9.1 Cómo leer logs (el comando que más vas a usar).....	57
9.2 Problemas de conexión SSH	58
9.3 Problemas con Docker	58
9.4 Problemas con	60
9.5 Problemas con el frontend.....	61
9.6 Problemas con el backend.....	62
9.7 Problemas de DNS y dominio	63

9.8 Comandos de diagnóstico que conviene memorizar.....	64
9.9 Reiniciar todo "porque sí" cuando estás bloqueado.....	64
9.10 Borrar cosas selectivamente sin cargarte la base de datos.....	65
9.11 Quedarse sin espacio en disco.....	70
9.12 Cuándo pedir ayuda.....	70
10 Migrar a un VPS nuevo	72
10.1 Conceptos previos: ¿qué hay que mover?	72
10.2 Estrategia general.....	72
10.3 Antes de empezar: una checklist	73
10.4 Paso 1: preparar el VPS nuevo	73
10.5 Paso 2: dumps de las bases de datos en el viejo.....	74
10.6 Paso 3: empaquetar todo lo que hay que mover	74
10.7 Paso 4: transferir al VPS nuevo	75
10.8 Paso 5: levantar todo en el VPS nuevo	76
10.9 Paso 6: probar antes de tocar DNS	77
10.10 Paso 7 (opcional): copiar volúmenes Docker enteros	78
10.11 Paso 8: el cambio de DNS (el momento de la verdad)	79
10.12 Paso 9: dar de baja el VPS viejo	81
10.13 Resumen visual del proceso	81

1 Despliegue en VPS con Docker

Guía paso a paso para alquilar un servidor, configurarlo desde cero y subir vuestros proyectos (frontend y backend) usando Docker, con HTTPS automático.

1.1 Para quién es esta guía

Para alumnos que **ya han usado la terminal** (saben moverse con `cd`, `ls`, instalar paquetes con `npm` ...) pero **nunca han trabajado con un servidor remoto**. No hace falta saber Linux, ni Docker, ni redes. Lo explicaremos a medida que aparezca.

1.2 Qué vais a conseguir al terminar

- Un servidor en internet con vuestro dominio (`https://midominio.com`).
- Capacidad para subir **varios proyectos a la vez** en el mismo servidor, cada uno en su subdominio (`api.midominio.com` , `app.midominio.com` ...).
- **HTTPS automático** y gratuito, sin tener que renovarlo a mano.
- Un flujo de actualización sencillo: cambias el código, lo subes, y los usuarios ven la nueva versión.

1.3 Cómo seguir la guía

Lee los capítulos **en orden la primera vez**. Cada uno depende del anterior. Cuando ya hayas desplegado tu primer proyecto, podrás saltar directamente a los capítulos 5 y 6 para subir proyectos nuevos.

Si te bloqueas, ve al capítulo de troubleshooting antes de pedir ayuda — el 80% de los problemas comunes están ahí.

2 Introducción y arquitectura general

Antes de empezar a teclear comandos, conviene entender **qué vamos a montar y por qué**. Si entiendes la foto general, los pasos individuales tendrán mucho más sentido.

2.1 El problema que queremos resolver

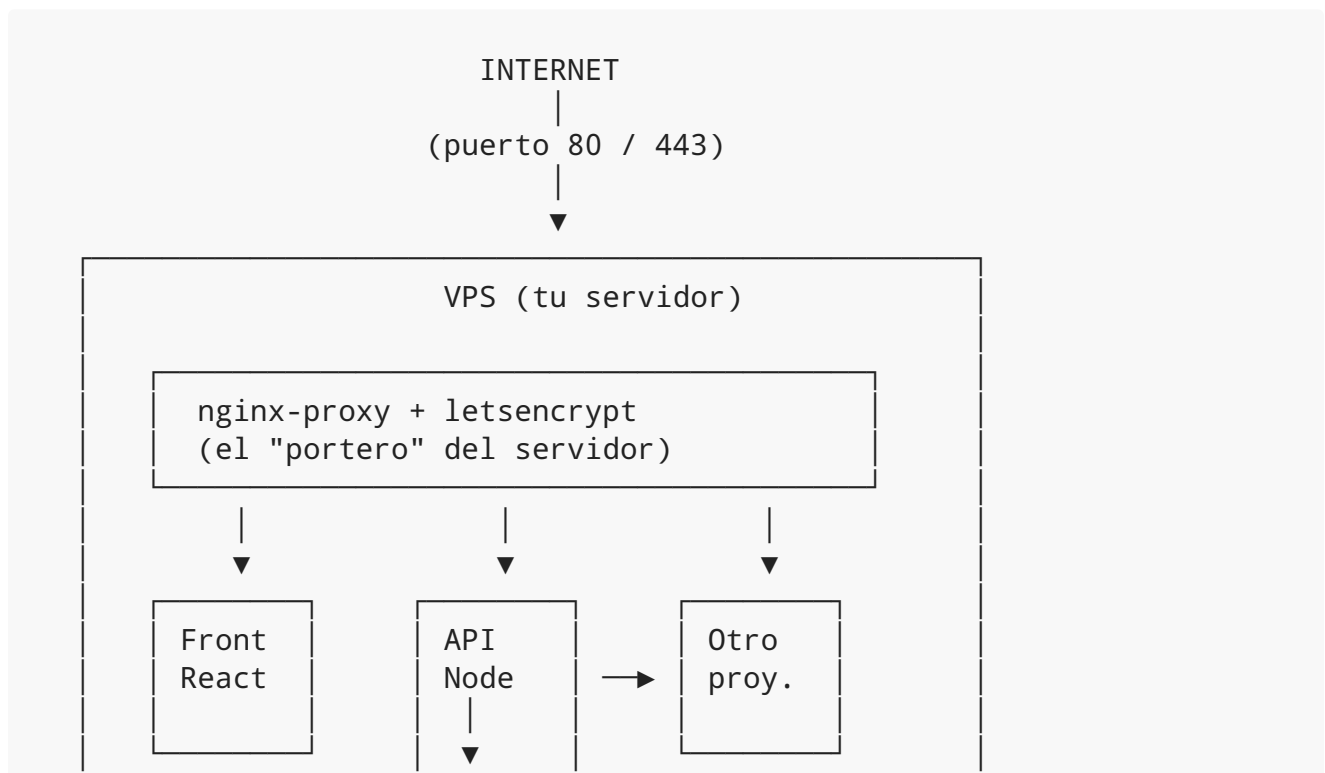
Cuando desarrollas en local, abres `http://localhost:3000` y todo funciona. Pero eso solo lo ves tú. Para que **cualquier persona del mundo** pueda entrar a tu aplicación necesitas:

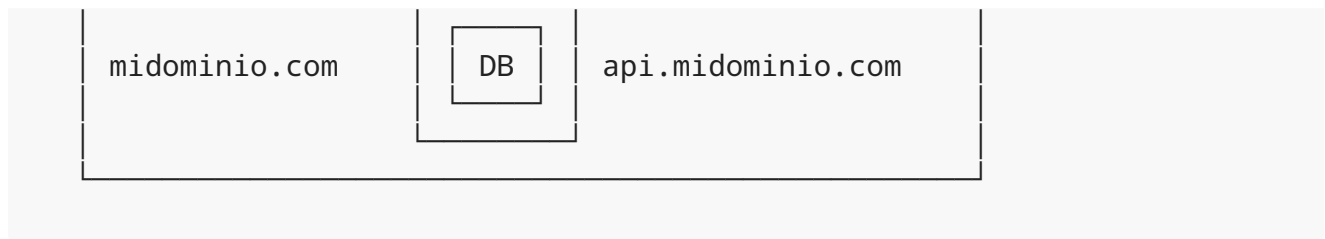
1. **Un ordenador encendido 24/7** conectado a internet con una IP pública → eso es un **VPS**.
2. **Un nombre fácil de recordar** que apunte a esa IP → eso es un **dominio + DNS**.
3. **Un programa que sirva tu app** cuando alguien entre por el navegador → eso es **nginx** (para el front) o tu propio servidor Node/Python/etc. (para el back).
4. **HTTPS** (el candadito verde) para que el navegador no muestre avisos de seguridad → eso es **Let's Encrypt**.
5. Y como vas a querer subir **varios proyectos al mismo servidor**, necesitas algo que decida qué petición va a qué proyecto → eso es un **reverse proxy** (en nuestro caso, `nginx-proxy`).

Esta guía monta todo eso.

2.2 La arquitectura final

Esta es la foto de lo que vamos a tener cuando terminemos:





Lectura del dibujo:

- Toda petición de internet llega al VPS por los puertos 80 (HTTP) y 443 (HTTPS).
- El **nginx-proxy** mira el dominio (`midominio.com` vs `api.midominio.com`) y reenvía la petición al contenedor correcto.
- Cada proyecto vive en su propio **contenedor Docker**, aislado del resto.
- La **base de datos** está en una red privada interna: solo el backend puede hablar con ella, no es accesible desde internet.

2.3 Por qué Docker

Podríamos instalar Node, MySQL, nginx... directamente en el servidor. Pero eso tiene problemas:

- **Conflictos de versiones:** el proyecto A necesita Node 18, el B necesita Node 20.
- **Difícil de replicar:** si el servidor se rompe, montarlo de nuevo es un infierno.
- **Difícil de aislar:** si un proyecto rompe algo del sistema, afecta a todos.

Con **Docker** cada proyecto se mete en una "caja" (contenedor) que trae dentro todo lo que necesita. La caja se puede crear, parar, borrar y volver a crear sin tocar el sistema. Y si pasas la "receta" (el `docker-compose.yml`) a otra persona, lo puede levantar igual en su máquina en 30 segundos.

2.4 El flujo de trabajo que vamos a aprender

Cuando termines la guía, el ciclo de "subir un cambio" será así:

```
# 1. Trabajas en local, haces tus cambios
# 2. Subes el código al servidor (git pull o scp)
# 3. Reconstruyes el contenedor
docker compose up -d --build
# 4. Listo, los usuarios ya ven la nueva versión
```

Eso es todo. Sin FTPs, sin "subir archivos uno a uno", sin reiniciar el servidor.

2.5 Lo que necesitas antes de empezar

- Una **tarjeta** para alquilar un VPS (los más baratos rondan los 3-5 €/mes).

- Un **dominio** comprado en cualquier registrador (OVH, IONOS, Namecheap, Porkbun...). Si tu bootcamp os proporciona subdominios, también vale.
- Un **email** real para Let's Encrypt (te notifica si un certificado va a caducar; en la práctica no te llegará nada porque se renueva solo).
- Tu **ordenador con terminal** y conexión a internet.

2.5.1 ¿Qué proveedor de VPS elijo?

Para esta guía sirve cualquiera. Algunas opciones razonables, con distintos perfiles:

Proveedor	Origen	Comentario
OVH	 Francia	Buena opción europea, datacenters en España.
IONOS	 Alemania	Muy buen soporte en español, panel sencillo.
Hetzner	 Alemania	Mejor relación calidad/precio en Europa.
DigitalOcean	 EE.UU.	Mucha documentación y tutoriales (en inglés).
Contabo	 Alemania	Mucha RAM por poco dinero, soporte más lento.

 **Consejo:** si nunca has alquilado un VPS, **OVH** o **IONOS** son buenas opciones si quieres factura en español y datos alojados en la UE (importante si vas a manejar datos de usuarios, por el RGPD). Cualquiera de los cinco sirve para esta guía: en todos pides "VPS con Ubuntu 22.04 o 24.04" y obtienes lo mismo.

Para empezar, pide la configuración más básica: **1 vCPU, 2 GB de RAM, 20-40 GB de disco** es más que suficiente para alojar varios proyectos pequeños.

En el siguiente capítulo veremos los **conceptos previos** que necesitas tener claros antes de tocar nada.

3 Conceptos previos

Este capítulo no tiene comandos. Es la "teoría mínima" que necesitas saber para que el resto de la guía no sea magia. Si ya conoces algún apartado, sáltatelo.

3.1 ¿Qué es un VPS?

VPS significa *Virtual Private Server* (servidor privado virtual). Es básicamente **un ordenador en internet** al que tú accedes en remoto. No tiene pantalla ni teclado: te conectas a él desde tu terminal.

Físicamente es una porción de un servidor grande en un centro de datos. La empresa que te lo alquila (DigitalOcean, Hetzner, etc.) te da:

- Una **IP pública** (por ejemplo `123.45.67.89`) → es la "dirección" del servidor en internet.
- Un **usuario y contraseña** (o una clave SSH) para entrar.
- Un sistema operativo instalado, normalmente **Ubuntu Linux**.

A partir de ahí el servidor es **tuyo**: puedes instalar lo que quieras, dejarlo encendido para siempre y servir aplicaciones a todo el mundo.

 **Analogía:** piensa en un VPS como en alquilar un piso pequeño en un edificio compartido. El edificio (la máquina física) es de la empresa, pero tu piso (tu VPS) es solo tuyo, con tu llave, y dentro haces lo que quieras.

3.2 ¿Qué es un dominio y cómo funcionan los DNS?

Los humanos recordamos `google.com`, no `142.250.184.46`. Los **dominios** son nombres legibles que apuntan a IPs. Quien hace esa traducción es el **DNS** (*Domain Name System*).

Cuando alguien escribe `midominio.com` en el navegador, pasa esto:

1. El navegador pregunta al DNS: "¿qué IP tiene `midominio.com`?"
2. El DNS responde: " `123.45.67.89` ".
3. El navegador conecta con esa IP y pide la página.

Para que **tu dominio apunte a tu VPS** tienes que ir al panel de tu registrador (donde compraste el dominio) y crear **registros DNS**.

3.2.1 Registros DNS que vas a necesitar

Tipo	Nombre	Valor	Significado
A	@	<code>123.45.67.89</code>	<code>midominio.com</code> → tu VPS

Tipo	Nombre	Valor	Significado
A	www	123.45.67.89	www.midominio.com → tu VPS
A	api	123.45.67.89	api.midominio.com → tu VPS
A	*	123.45.67.89	cualquier subdominio → tu VPS (wildcard)

 **Importante:** los cambios de DNS **tardan en propagarse** (de 5 minutos a unas horas, normalmente 15-30 min). Si configuras un dominio y "no funciona", suele ser eso. Puedes comprobarlo con `dig midominio.com` o en dnschecker.org.

3.2.2 Añadir un subdominio para un proyecto nuevo

Cada vez que quieras publicar un proyecto nuevo en un subdominio distinto, el proceso es siempre el mismo:

1. Añade el registro DNS en tu registrador apuntando a la misma IP del VPS:

Tipo	Nombre	Valor
A	blog	123.45.67.89

Esto hace que `blog.midominio.com` llegue a tu servidor.

2. Espera a que propague (10-30 minutos). Compruébalo:

```
dig +short blog.midominio.com
# Debe devolver tu IP
```

3. Levanta el contenedor con `VIRTUAL_HOST=blog.midominio.com` (lo veremos en los capítulos de despliegue). `nginx-proxy` lo detectará automáticamente y `acme-companion` pedirá el certificado HTTPS.

No hay que tocar la configuración del proxy ni reiniciar nada. El DNS y el `VIRTUAL_HOST` son todo lo que necesitas.

💡 **Truco del wildcard:** si en tu registrador creas un registro `A` con nombre `*` apuntando a tu VPS, **cualquier subdominio** que inventes llegará al servidor sin tener que crear un registro nuevo cada vez. Solo tienes que levantar el contenedor con el `VIRTUAL_HOST` correcto. Útil si vas a desplegar muchos proyectos. La única limitación es que Let's Encrypt no puede emitir certificados wildcard con el método HTTP estándar (necesitaría DNS-01, que es más complejo). Para la mayoría de proyectos de bootcamp, crear los registros uno a uno es más que suficiente.

3.2.3 ¿Puedo usar el mismo dominio raíz y un subdominio a la vez?

Sí, y es el caso más habitual:

- `midominio.com` → frontend React.
- `api.midominio.com` → backend Node.

Ambos registros apuntan a la misma IP. Es `nginx-proxy` quien diferencia a cuál de los dos contenedores va cada petición, mirando el dominio que pide el navegador.

3.3 ¿Qué es SSH?

SSH (*Secure Shell*) es el protocolo estándar para **abrir una terminal en una máquina remota** de forma segura. Cuando haces:

```
ssh ubuntu@123.45.67.89
```

estás diciendo: "abre una sesión de terminal en la máquina `123.45.67.89` como el usuario `ubuntu`". A partir de ahí cualquier comando que escribas se ejecuta **allí**, no en tu ordenador.

3.3.1 Por qué claves SSH y no contraseñas

Puedes conectarte con contraseña, pero es mala idea:

- Es vulnerable a ataques de fuerza bruta (hay bots que prueban contraseñas 24/7 contra cualquier IP).
- Es incómodo (tienes que escribirla cada vez).

La alternativa es la **autenticación por clave pública/privada**:

- Generas un par de claves en tu ordenador: una **privada** (que NUNCA sale de tu máquina) y una **pública** (que puedes compartir sin riesgo).
- Copias la **pública** al servidor.
- A partir de entonces te conectas sin contraseña: el servidor verifica que tienes la privada que corresponde.

🔑 **Analogía:** la clave pública es una **cerradura** que pegas en una puerta. Cualquiera puede ver la cerradura. La clave privada es la **única llave** que la abre. La tienes solo tú.

3.4 ¿Qué es Docker?

Docker es una herramienta que mete cada aplicación en una **caja aislada** llamada **contenedor**. Dentro de la caja viven: el código, las dependencias, el sistema operativo mínimo... todo lo que la aplicación necesita.

3.4.1 Tres conceptos que debes distinguir

Concepto	Qué es	Analogía
Imagen	Una "plantilla" estática con todo lo necesario para arrancar la aplicación.	Un molde de galletas.
Contenedor	Una instancia en ejecución de una imagen. De una imagen puedes lanzar muchos contenedores.	La galleta ya horneada y comiéndose.
Volumen	Una carpeta persistente que sobrevive a borrar contenedores (para datos como bases de datos).	El plato donde dejas la galleta.

3.4.2 ¿Qué es `docker compose` ?

Si tu aplicación tiene **varios servicios** (backend + base de datos + nginx...), levantar cada contenedor a mano es tedioso. **Docker Compose** te permite describir todos los servicios en **un solo archivo YAML** (`docker-compose.yml`) y arrancarlos juntos con un comando:

```
docker compose up -d
```

Es la herramienta principal que vamos a usar.

i Verás `docker-compose` (con guión) en guías antiguas y `docker compose` (con espacio) en las modernas. Es lo mismo, pero la versión moderna está integrada en Docker y es la que tendrás tú. Usa siempre `docker compose` con espacio.

3.5 ¿Qué es un reverse proxy y por qué lo necesitamos?

Un **reverse proxy** es un programa que recibe peticiones de internet y las **reenvía al servicio interno adecuado**. Es el "portero" de tu servidor.

3.5.1 El problema sin reverse proxy

Imagina que tienes dos proyectos en el mismo VPS:

- Un frontend que necesita servir en el puerto 80.
- Una API que también necesita el puerto 80.

No pueden estar los dos en el puerto 80 a la vez. ¿Y si añades un tercero? ¿Y un cuarto?

3.5.2 La solución con `nginx-proxy`

`nginx-proxy` es el único que escucha los puertos 80 y 443. Cuando llega una petición, mira el **dominio** que pide el navegador:

- ¿Pide `midominio.com` ? → la manda al contenedor del frontend.
- ¿Pide `api.midominio.com` ? → la manda al contenedor del backend.
- ¿Pide `blog.midominio.com` ? → la manda al contenedor del blog.

Y la magia: cada contenedor solo necesita declarar **a qué dominio responde** (con una variable `VIRTUAL_HOST`) y el proxy lo detecta automáticamente. No tienes que reconfigurar nada cuando añades un proyecto nuevo.

3.6 ¿Qué es Let's Encrypt y por qué necesito HTTPS?

HTTPS es la versión cifrada de HTTP. Para que un sitio funcione con HTTPS necesita un **certificado SSL/TLS** firmado por una entidad de confianza.

Let's Encrypt es una entidad que **emite certificados gratis**, válidos por 90 días y renovables automáticamente. Junto con `nginx-proxy` usaremos un compañero llamado `acme-companion` que:

1. Detecta los dominios que declaras en tus contenedores.
2. Pide los certificados a Let's Encrypt.
3. Los renueva automáticamente antes de que caduquen.

Tú no tienes que hacer absolutamente nada después de la configuración inicial. Es uno de los pasos más mágicos de toda la guía.



¿Por qué necesito HTTPS si mi proyecto es de juguete? Porque sin HTTPS:

- Los navegadores modernos muestran avisos rojos de "sitio inseguro".
- Muchas APIs y funciones del navegador (geolocalización, micrófono, cámara, *service workers*...) **solo funcionan con HTTPS**.
- Google te penaliza en buscador.
- Es gratis. No hay excusa.

Con estos conceptos en la cabeza, ya puedes pasar al l o. En el siguiente cap tulo nos conectamos al servidor por primera vez.

4 Conexión al servidor y claves SSH

En este capítulo nos conectamos al VPS por primera vez y configuramos las claves SSH para no tener que usar contraseñas. Lo haremos en dos partes:

1. **Conexión inicial** (con la contraseña que te dio el proveedor).
 2. **Configurar claves SSH** para entrar sin contraseña.
 3. **Opcional pero recomendado:** configurar SSH en el servidor para conectarse a GitHub desde allí.
-

4.1 Obtener los datos de acceso

Cuando contratas un VPS, el proveedor te da estos datos:

- **IP pública** del servidor, por ejemplo `123.45.67.89`.
- **Usuario** inicial, normalmente `root` o `ubuntu`.
- **Contraseña** o, si elegiste autenticación con clave durante la creación, la clave que configuraste.

Apunta esto antes de seguir. En la guía usaremos los placeholders `<usuario>` y `<host>` (el host es la IP o un dominio).

4.2 Primera conexión

Abre una terminal en tu ordenador y conecta:

```
ssh <usuario>@<host>
```

Ejemplo real:

```
ssh ubuntu@123.45.67.89
```

La primera vez te aparecerá un mensaje como este:

```
The authenticity of host '123.45.67.89' can't be established.  
ED25519 key fingerprint is SHA256:abc123...  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Escribe `yes` y pulsa Enter. Esto guarda la "huella" del servidor para que tu cliente SSH te avise si en el futuro la huella cambia (señal de un posible ataque *man-in-the-middle*).

Te pedirá la contraseña. Escríbela (no verás los caracteres mientras tecleas, es normal) y pulsa Enter.

Si todo va bien verás algo así:

```
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-1041-aws x86_64)
...
ubuntu@servidor:~$
```

¡Ya estás dentro del servidor! Todo lo que escribas a partir de aquí se ejecuta en el VPS. Para salir, escribe `exit` o pulsa `Ctrl + D`.

4.2.1 ¿Y si "Permission denied"?

- Revisa que el usuario sea el correcto (algunos proveedores usan `root`, otros `ubuntu`, otros `debian`).
- Revisa que la IP sea la correcta.
- Si configuraste autenticación por clave al crear el VPS, no podrás entrar con contraseña: tendrás que indicar la clave con `ssh -i ~/.ssh/<tu_clave> <usuario>@<host>`.

4.3 Generar una clave SSH en tu ordenador

Ahora vamos a configurar la autenticación por clave para no escribir contraseña cada vez. **Abre una NUEVA terminal en tu ordenador local** (no en el servidor) y ejecuta:

```
ssh-keygen -t ed25519 -C "tu-email@ejemplo.com"
```

Desglose:

- `ssh-keygen` : el comando que genera claves.
- `-t ed25519` : el tipo de cifrado (Ed25519 es el moderno, más seguro y rápido que el clásico RSA).
- `-C "..."` : un **comentario** para identificar la clave. Suele usarse el email o algo como `"laptop-personal-vps-bootcamp"`. No afecta a la seguridad, solo te ayuda a saber qué clave es cuál cuando tengas varias.

Te hará dos preguntas:

```
Enter file in which to save the key (/home/tu-user/.ssh/id_ed25519):
```


Si pulsas Enter, la guarda como `id_ed25519` (la clave por defecto). **Recomendado:** dale un nombre descriptivo para no mezclarlas:

```
/home/tu-user/.ssh/id_ed25519_vps_bootcamp
```

Luego:

```
Enter passphrase (empty for no passphrase):
```

La *passphrase* es una contraseña que **cifra tu clave privada**. Si alguien te roba el archivo de la clave, no podrá usarla sin la passphrase. Es **muy recomendable** ponerla. Pulsa Enter dos veces si no quieres usarla (no recomendado en máquinas compartidas).

Resultado: ahora tienes dos archivos en `~/.ssh/` :

- `id_ed25519_vps_bootcamp` → la **clave privada**. **JAMÁS** la compartas, ni la subas a Git, ni la mandes por chat.
- `id_ed25519_vps_bootcamp.pub` → la **clave pública**. Esta se copia al servidor.

4.4 Copiar la clave pública al servidor

Hay una utilidad que lo hace por ti:

```
ssh-copy-id -i ~/.ssh/id_ed25519_vps_bootcamp.pub <usuario>@<host>
```

Te pedirá la contraseña del servidor **una última vez**. A partir de ahora podrás entrar sin contraseña.

4.4.1 ¿Qué ha hecho ese comando exactamente?

Ha copiado el contenido de tu clave pública al archivo `~/.ssh/authorized_keys` del servidor. Ese archivo contiene la lista de "cerraduras" que el servidor acepta. Cualquier ordenador que tenga la clave privada correspondiente a una de esas cerraduras puede entrar.

4.4.2 Si `ssh-copy-id` no funciona (Windows sin WSL)

Conéctate manualmente y pega la clave a mano:

```
# En tu ordenador, copia el contenido de la clave pública
cat ~/.ssh/id_ed25519_vps_bootcamp.pub
# Cópialo al portapapeles

# Conéctate al servidor con contraseña
ssh <usuario>@<host>

# En el servidor:
mkdir -p ~/.ssh
chmod 700 ~/.ssh
echo "PEGA_AQUÍ_LA_CLAVE_PÚBLICA" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

4.5 Configurar un alias en `~/.ssh/config`

Escribir `ssh -i ~/.ssh/id_ed25519_vps_bootcamp ubuntu@123.45.67.89` cada vez es un coñazo. Vamos a configurar un alias.

En tu ordenador, abre (o crea) el archivo `~/.ssh/config`:

```
nano ~/.ssh/config
```

Y añade:

```
Host mi-vps
  HostName 123.45.67.89
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519_vps_bootcamp
```

Desglose:

- `Host mi-vps` → el alias que vamos a usar. Puede ser cualquier nombre.
- `HostName` → la IP o dominio real.
- `User` → el usuario con el que te conectas.
- `IdentityFile` → la ruta a tu clave privada.

Guarda (`Ctrl + O`, `Enter`, `Ctrl + X` en nano) y ahora puedes conectarte así:

```
ssh mi-vps
```

Y ya está. Sin IP, sin usuario, sin contraseña, sin clave a mano. Esto también te permite conectarte con **VS Code** usando la extensión "Remote - SSH": en lugar de configurar todo en la extensión, te detectará `mi-vps` automáticamente.

4.6 (Opcional) Configurar SSH en el servidor para usar GitHub

Si vas a desplegar el backend, vas a clonar repositorios desde el servidor. Si tus repos son **privados**, GitHub no te dejará clonarlos sin autenticación. La forma limpia es generar una clave SSH **dentro del servidor** y registrarla en GitHub.

Conéctate al servidor:

```
ssh mi-vps
```

Genera la clave dentro del servidor:

```
ssh-keygen -t ed25519 -C "vps-bootcamp"  
# Pulsa Enter para aceptar la ruta por defecto  
# Pon (o no) passphrase
```

Muestra la clave pública:

```
cat ~/.ssh/id_ed25519.pub
```

Te imprimirá algo como:

```
ssh-ed25519 AAAAC3Nza... vps-bootcamp
```

Copia esa línea completa.

Pégala en GitHub:

1. Entra a github.com/settings/keys.
2. Pulsa *New SSH key*.
3. Dale un título (`mi-vps-bootcamp`), pega la clave en el campo `Key` y guarda.

Comprueba que funciona (sigues en el servidor):

```
ssh -T git@github.com
```

Debería responder:

```
Hi tu-usuario! You've successfully authenticated, but GitHub does not
provide shell access.
```

A partir de ahora puedes clonar tus repos privados con `git clone git@github.com:usuario/repo.git` desde el servidor.

Ya tienes acceso fluido al servidor. En el siguiente capítulo instalamos Docker.

5 Instalación de Docker en el servidor

En este capítulo instalamos Docker y Docker Compose en el VPS. Vamos a instalar la versión oficial directamente desde el repositorio de Docker (no la del repositorio de Ubuntu, que suele estar más desactualizada).



Conéctate al servidor antes de empezar: `ssh mi-vps`. Todos los comandos de este capítulo se ejecutan **dentro del VPS**, no en tu máquina local.

5.1 Actualizar el sistema

Antes de instalar nada, es buena costumbre actualizar la lista de paquetes y aplicar las actualizaciones de seguridad pendientes:

```
sudo apt update
sudo apt upgrade -y
```

Desglose:

- `sudo` → ejecuta el comando como administrador. Te pedirá la contraseña del usuario la primera vez en cada sesión.
- `apt update` → refresca la lista de paquetes disponibles. **No instala nada todavía.**
- `apt upgrade -y` → instala las actualizaciones disponibles. El `-y` responde "sí" automáticamente a todas las preguntas.

Si te pregunta sobre archivos de configuración modificados, pulsa Enter para mantener la versión actual (por defecto).



La primera vez puede tardar varios minutos. Es normal.

5.2 Instalar Docker (paso a paso)

Estos pasos vienen de la [documentación oficial de Docker para Ubuntu](#). Te los explico al detalle.

5.2.1 Paso 1: instalar prerequisites

```
sudo apt-get install -y ca-certificates curl gnupg
```

Estos paquetes son los que permiten descargar e instalar Docker de forma segura:

- `ca-certificates` → certificados raíz para validar conexiones HTTPS.
- `curl` → herramienta para descargar archivos desde URLs.
- `gnupg` → utilidad para verificar firmas criptográficas (vamos a verificar que Docker es auténtico).

5.2.2 Paso 2: añadir la clave GPG oficial de Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

¿Qué hace esto? Descarga la clave pública con la que Docker firma sus paquetes y la guarda en `/etc/apt/keyrings/`. A partir de ahora `apt` solo aceptará paquetes firmados con esta clave (garantiza que no instalas un Docker manipulado).

5.2.3 Paso 3: añadir el repositorio de Docker a apt

```
echo \
"deb [arch=\"$(dpkg --print-architecture)\" signed-by=/etc/apt/
keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
\"$(. /etc/os-release && echo \"$VERSION_CODENAME\")\" stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Parece intimidante pero es un copy-paste. Lo que hace: añade el repositorio oficial de Docker a la lista de fuentes de `apt`, detectando automáticamente la arquitectura de tu CPU (`amd64`, `arm64` ...) y la versión de Ubuntu que tengas.

5.2.4 Paso 4: actualizar la lista de paquetes (ahora con el repo de Docker añadido)

```
sudo apt-get update
```

5.2.5 Paso 5: instalar Docker

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-  
buildx-plugin docker-compose-plugin
```

Esto instala:

- `docker-ce` → el motor de Docker.
- `docker-ce-cli` → el comando `docker` para hablar con el motor.
- `containerd.io` → el componente de bajo nivel que gestiona los contenedores.
- `docker-buildx-plugin` → para construir imágenes de forma avanzada.
- `docker-compose-plugin` → el subcomando `docker compose` (con espacio).

5.3 Verificar que Docker funciona

```
sudo docker run hello-world
```

Si todo está bien, Docker descargará una imagen de prueba, la ejecutará y verás:

```
Hello from Docker!  
This message shows that your installation appears to be working  
correctly.  
...
```

Si ves esto, **enhorabuena, Docker está instalado.**

5.4 Permitir ejecutar Docker sin `sudo`

Habrás notado que hemos puesto `sudo` delante de `docker run`. Eso es porque por defecto solo el usuario `root` puede usar Docker. Es incómodo y, en scripts, problemático.

Vamos a añadir tu usuario al **grupo** `docker` para evitar el `sudo`:

```
sudo groupadd docker          # crea el grupo (si no existe; puede dar  
"already exists", no pasa nada)  
sudo usermod -aG docker $USER # añade tu usuario actual al grupo  
newgrp docker                 # activa el cambio en la sesión actual sin  
tener que reloguear
```

⚠ **Aviso de seguridad:** pertenecer al grupo `docker` equivale, en la práctica, a tener permisos de `root` (porque desde Docker puedes acceder al sistema entero). No añadas a este grupo a usuarios en los que no confíes plenamente.

Verifica que funciona sin `sudo` :

```
docker run hello-world
```

Si te sigue dando "permission denied", **desconéctate y vuelve a conectarte por SSH**:

```
exit
ssh mi-vps
docker run hello-world
```

5.5 Limpiar las pruebas

Hemos creado un par de contenedores de prueba con `hello-world` . No molestan, pero podemos limpiarlos:

```
docker container prune -f # borra contenedores parados
docker image prune -a -f  # borra imágenes que no se estén usando
```

5.6 Comandos básicos de Docker que vas a usar todo el rato

Comando	Qué hace
<code>docker ps</code>	Lista los contenedores en ejecución .
<code>docker ps -a</code>	Lista todos los contenedores (incluso parados).
<code>docker images</code>	Lista las imágenes que tienes descargadas.
<code>docker logs <nombre></code>	Muestra los logs de un contenedor.
<code>docker logs -f <nombre></code>	Muestra los logs en vivo (sigue actualizando).
<code>docker exec -it <nombre> bash</code>	Te mete dentro del contenedor (como un SSH).

Comando	Qué hace
<code>docker compose up -d</code>	Arranca todos los servicios del <code>docker-compose.yml</code> .
<code>docker compose down</code>	Para y borra los contenedores del compose.
<code>docker compose logs -f</code>	Logs en vivo de todos los servicios del compose.
<code>docker compose ps</code>	Lista los contenedores del compose actual.

Memoriza los dos comandos clave: `docker ps` y `docker compose logs -f`. Son los que más vas a usar.

Con Docker instalado y funcionando, el siguiente paso es montar el **nginx-proxy** que dirigirá el tráfico a tus proyectos y gestionará el HTTPS.

6 Configurar nginx-proxy y HTTPS automático

En este capítulo levantamos el **portero** del servidor: dos contenedores (`nginx-proxy` y su compañero de Let's Encrypt) que se quedarán encendidos para siempre. Una vez configurados, cualquier proyecto nuevo que añadas obtendrá automáticamente:

- Acceso desde su dominio.
- Un certificado HTTPS gratuito.
- Renovación automática del certificado.

Esto es el **corazón** de la configuración. Lo haces una vez por servidor.

6.1 Asegurar que los puertos 80 y 443 están abiertos

`nginx-proxy` necesita escuchar en los puertos **80** (HTTP) y **443** (HTTPS). Por defecto en la mayoría de VPS están abiertos, pero compruébalo:

6.1.1 Si usas el firewall `ufw` (Ubuntu)

```
sudo ufw status
```

Si está activo y los puertos 80/443 no aparecen como "ALLOW", ábrelos:

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow OpenSSH           # ¡no te olvides de SSH, o te quedas fuera!
```

6.1.2 Si tu proveedor tiene firewall propio (DigitalOcean, AWS, Hetzner Cloud Firewall...)

Entra al panel del proveedor y abre los puertos 80, 443 y 22 (SSH). Cada proveedor tiene su interfaz, búscalos como "Firewall", "Cloud Firewall" o "Security Groups".

 **Importante:** si cierras el puerto 22 sin querer, **te quedas fuera del servidor**. Siempre verifica que SSH está permitido antes de activar un firewall.

6.2 Crear la red de Docker para el proxy

Todos los contenedores que quieran ser accesibles por internet (frontends, backends...) tienen que estar en la **misma red Docker** que `nginx-proxy`. Vamos a crear esa red.

```
docker network create nginx-proxy
```

Verifica que se ha creado:

```
docker network ls
```

Deberías ver `nginx-proxy` en la lista.



¿Por qué una red separada? En Docker, los contenedores solo pueden hablar entre sí si están en la **misma red**. Esto nos permite tener proyectos aislados (cada uno en su red privada) pero todos conectados al portero (`nginx-proxy`) por una red común.

6.3 Crear el `docker-compose.yml` del proxy

Vamos a crear una carpeta dedicada al proxy y meterle su `docker-compose.yml`. La convención que vamos a seguir en toda la guía es organizar todo bajo `/home/<usuario>/proyectos/`.

```
mkdir -p ~/proyectos/nginx-proxy
cd ~/proyectos/nginx-proxy
nano docker-compose.yml
```

Pega esto dentro:

```
services:
  nginx-proxy:
    image: nginxproxy/nginx-proxy
    container_name: nginx-proxy
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - conf:/etc/nginx/conf.d
      - ./vhost.d:/etc/nginx/vhost.d      # carpeta local, editable sin
```

```

reiniciar
  - dhparam:/etc/nginx/dhparam
  - certs:/etc/nginx/certs:ro
  - html:/usr/share/nginx/html
  - /var/run/docker.sock:/tmp/docker.sock:ro
networks:
  - proxy
restart: always

letsencrypt:
  image: nginxproxy/acme-companion
  container_name: nginx-proxy-le
  volumes_from:
    - nginx-proxy
  volumes:
    - certs:/etc/nginx/certs:rw
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - acme:/etc/acme.sh
  environment:
    - DEFAULT_EMAIL=tu@email.com          # ← pon tu email aquí
  restart: always

volumes:
  conf:
  dhparam:
  certs:
  acme:
  html:

networks:
  proxy:
    name: nginx-proxy
    external: true

```

Guarda con `Ctrl + O`, Enter, y sal con `Ctrl + X`.

💡 Fíjate en que el volumen `vhost.d` apunta a `./vhost.d` (una carpeta local en el servidor), no a un volumen Docker nombrado. Esto es intencional: te permite editar los archivos de configuración directamente, que sobreviven a cualquier `docker compose up` y puedes versionar en git. Lo veremos en detalle más adelante.

6.3.1 Desglose del archivo

Servicio `nginx-proxy`:

```
image: nginxproxy/nginx-proxy
```

Imagen oficial y mantenida activamente. Es el sucesor de la antigua `jwilder/nginx-proxy`. Hace una sola cosa: leer los contenedores que arrancas, ver cuál tiene `VIRTUAL_HOST` definido y reenviarles el tráfico.

```
ports:
  - "80:80"
  - "443:443"
```

Expone los puertos al exterior. Formato `host:contenedor`: el puerto 80 del servidor → el puerto 80 del contenedor.

```
- /var/run/docker.sock:/tmp/docker.sock:ro
```

El truco mágico. Le da al proxy acceso (en solo lectura, `:ro`) al socket de Docker. Así puede "ver" los demás contenedores y descubrir cuáles tienen `VIRTUAL_HOST`. Sin esto, no funcionaría nada.

Servicio `letsencrypt`:

```
image: nginxproxy/acme-companion
```

El compañero que pide y renueva certificados. Detecta contenedores con `LETSENCRYPT_HOST` definido y se encarga del proceso automáticamente.

```
volumes_from:
  - nginx-proxy
```

Comparte los volúmenes con `nginx-proxy` (necesita guardar los certificados donde el proxy pueda leerlos).

Sección `networks`:

```
networks:
  proxy:
```

```
name: nginx-proxy
external: true
```

Le dice a Docker: "usa la red `nginx-proxy` que ya existe (la que creamos antes), no crees una nueva".

6.4 Crear la carpeta `vhost.d`

Antes de levantar el proxy, crea la carpeta que montamos como volumen:

```
mkdir -p ~/proyectos/nginx-proxy/vhost.d
```

Esta carpeta es donde pondrás los ajustes personalizados de nginx para cada dominio (o para todos a la vez). La veremos en detalle en la sección siguiente.

6.5 Levantar el proxy

Desde la carpeta `~/proyectos/nginx-proxy`:

```
docker compose up -d
```

Desglose:

- `up` → crea y arranca los contenedores.
- `-d` → "detached", en segundo plano. Sin esto, la terminal se queda enganchada mostrando logs.

La primera vez tarda un poco (descarga las imágenes). Cuando termine, verifica:

```
docker compose ps
```

Deberías ver los dos contenedores con estado `running` o `up`:

NAME	IMAGE	STATUS	PORTS
nginx-proxy	nginxproxy/nginx-proxy	Up 30 seconds	

```
0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp
nginx-proxy-le      nginxproxy/acme-companion      Up 30 seconds
```

6.6 Comprobar que el proxy responde

Abre el navegador y ve a `http://<IP_DE_TU_VPS>`. Deberías ver una página de error de nginx ("503 Service Temporarily Unavailable" o similar). **Eso es buena señal:** significa que `nginx-proxy` está escuchando, pero todavía no hay ningún contenedor con `VIRTUAL_HOST` configurado al que mandar el tráfico.

Si no ves nada (timeout) → los puertos 80/443 están cerrados. Revisa el firewall (paso 4.1).

6.7 Ver los logs del proxy

Útil para depurar problemas con certificados:

```
# Los dos servicios:
docker compose logs -f

# Solo uno:
docker compose logs -f letsencrypt
```

Sal del modo "seguir logs" con `Ctrl + C` (no apaga el contenedor, solo deja de mostrar logs).

6.8 Configurar tu dominio en el DNS

Antes de desplegar nada, configura los registros DNS de tu dominio para que apunten al VPS. Ve al panel de tu registrador (donde compraste el dominio) y crea:

Tipo	Nombre	Valor
A	@	<IP_DE_TU_VPS>
A	www	<IP_DE_TU_VPS>
A	api	<IP_DE_TU_VPS>

 Los cambios DNS tardan en propagarse. **Espera 15-30 minutos** antes de intentar desplegar un proyecto, o Let's Encrypt fallará al pedir el certificado.

Para comprobar que ha propagado:

```
dig +short midominio.com
```

Debería devolvarte tu IP. Si no, espera más.

6.9 Personalizar la configuración de nginx (timeouts, tamaño de subida y más)

Una vez tienes el proxy funcionando, es muy probable que en algún momento necesites ajustar cómo se comporta nginx con alguno de tus proyectos: un backend que tarda en responder, una ruta de subida de archivos, etc.

`nginx-proxy` permite personalizar la configuración **por dominio** poniendo archivos en la carpeta `vhost.d/` que montamos antes. El archivo debe llamarse exactamente igual que el `VIRTUAL_HOST` del servicio.

6.9.1 Estructura

```
~/proyectos/nginx-proxy/  
└─ vhost.d/  
    ├── api.midominio.com      # ajustes solo para el backend  
    ├── midominio.com         # ajustes solo para el frontend  
    └─ default                 # ajustes para TODOS los dominios
```

6.9.2 Cómo crear un archivo de configuración por dominio

```
nano ~/proyectos/nginx-proxy/vhost.d/api.midominio.com
```

Y aplica los cambios sin reiniciar el contenedor:

```
docker exec nginx-proxy nginx -s reload
```


6.9.3 Casos de uso habituales

6.9.3.1 Aumentar el timeout (APIs lentas o llamadas a LLMs)

Por defecto nginx corta la conexión si el backend no responde en **60 segundos**. Si tu backend llama a una IA u otro servicio lento, necesitarás más tiempo:

```
proxy_read_timeout 300s;      # tiempo máximo esperando respuesta del
backend
proxy_connect_timeout 10s;    # tiempo para establecer la conexión
proxy_send_timeout 300s;      # tiempo enviando datos al backend
```

Pon el valor que necesites. Para LLMs con respuestas muy largas, 300s (5 minutos) suele ser suficiente.

6.9.3.2 Aumentar el tamaño máximo de subida de archivos

Por defecto nginx solo permite subir archivos de hasta **1 MB**. Si tu app acepta imágenes, PDFs o vídeos, necesitarás aumentarlo:

```
client_max_body_size 50m;      # permite subidas de hasta 50 MB
```

Ponlo a `0` si quieres desactivar el límite completamente (no recomendado en producción).

6.9.3.3 Activar streaming (respuestas en tiempo real)

Útil cuando el backend envía datos progresivamente (Server-Sent Events, respuestas de LLMs token a token...):

```
proxy_buffering off;
proxy_cache off;
proxy_set_header Connection '';
proxy_http_version 1.1;
```

Con `proxy_buffering off`, nginx reenvía los datos al cliente según los recibe, sin esperar a acumular la respuesta completa. Esto también evita timeouts en conexiones largas con actividad continua.

6.9.3.4 Ejemplo completo para un backend con LLM

Un archivo `vhost.d/api.midominio.com` típico para un backend que usa un LLM quedaría así:

```
proxy_read_timeout 300s;
proxy_connect_timeout 10s;
proxy_send_timeout 300s;
proxy_buffering off;
proxy_cache off;
client_max_body_size 20m;
```

6.9.4 Verificar que nginx ha cargado los cambios

Después de cada modificación:

```
# Recarga la configuración (sin reiniciar)
docker exec nginx-proxy nginx -s reload

# Verifica que los cambios están activos
docker exec nginx-proxy nginx -T | grep -A 10 "midominio.com"
```

Si ves tus directivas en la salida, está funcionando. Si no aparecen, revisa que el nombre del archivo coincide exactamente con el `VIRTUAL_HOST`.

6.9.5 Los cambios sobreviven a `docker compose up`

Al estar en una carpeta local (`./vhost.d`), los archivos de configuración no se pierden cuando reinicias o actualizas los contenedores. Es una de las ventajas de montar la carpeta como volumen local en vez de usar un volumen Docker nombrado. Puedes incluso añadirlos a tu repositorio git junto con el `docker-compose.yml` del proxy.

¡Tienes el proxy montado! A partir de aquí, **subir un proyecto consiste simplemente en levantar un contenedor con `VIRTUAL_HOST` y `LETSENCRYPT_HOST` definidos**. El resto sucede automáticamente.

En los siguientes dos capítulos lo vemos en la práctica: primero un frontend, luego un backend.

7 Desplegar un frontend (React / Vite)

En este capítulo desplegamos una **aplicación React** (Vite o Create-React-App, da igual) en el VPS. El frontend, una vez compilado, no es más que un puñado de archivos estáticos (HTML, CSS, JS) que vamos a servir con nginx.

Lo que vamos a hacer:

1. Compilar la app en tu ordenador.
2. Subir los archivos al servidor.
3. Crear un `docker-compose.yml` que arranque un nginx sirviendo esos archivos.
4. Verificar que se accede por `https://midominio.com`.

7.1 Compilar la app en local

En tu ordenador, dentro del proyecto React:

```
# Instala dependencias (si no lo has hecho ya)
npm install

# Compila para producción
npm run build
```

Esto genera una carpeta con los archivos listos para servir:

- Si usas **Vite**, se llama `dist/`.
- Si usas **Create-React-App**, se llama `build/`.

 Echa un vistazo dentro. Verás un `index.html`, una carpeta `assets/` con JS y CSS minificados, etc. **Eso** es lo que vamos a subir al servidor. No hace falta llevar `node_modules`, ni `src/`, ni nada más.

7.1.1 Variables de entorno en el frontend

Si tu app usa variables de entorno (por ejemplo `VITE_API_URL`), tienes que configurarlas **antes** de `npm run build`, porque se quedan "horneadas" en el bundle.

Por ejemplo, con Vite, crea un `.env.production` en local:

```
VITE_API_URL=https://api.midominio.com
```

Y luego compila. El valor queda en el JS compilado.

7.2 Estructura de carpetas en el servidor

Conéctate al servidor (`ssh mi-vps`) y crea la estructura del proyecto:

```
mkdir -p ~/proyectos/mi-app-front/web  
cd ~/proyectos/mi-app-front
```

La estructura final será:

```
~/proyectos/mi-app-front/  
├── docker-compose.yml      # configuración del contenedor  
├── web/                    # aquí van los archivos compilados  
│   ├── index.html  
│   ├── assets/  
│   └── ...  
└── nginx/                  # (opcional) configuración custom de nginx  
    └── nginx.conf
```

7.3 Subir los archivos compilados al servidor

Desde tu ordenador local (no desde el servidor), usa `scp` para copiar la carpeta `dist/` (o `build/`):

```
scp -r dist/* mi-vps:~/proyectos/mi-app-front/web/
```

Desglose:

- `scp` → "secure copy", copia archivos por SSH.
- `-r` → recursivo (copia carpetas enteras).
- `dist/*` → todos los archivos y subcarpetas dentro de `dist`.
- `mi-vps:~/proyectos/...` → destino. `mi-vps` es el alias que configuraste en `~/.ssh/config`.

💡 Si en lugar de `mi-vps` usas la IP directamente: `scp -r dist/* ubuntu@123.45.67.89:~/proyectos/mi-app-front/web/`.

Si tu carpeta de compilación se llama `build`, sustitúyelo: `scp -r build/* ...`.

Verifica desde el servidor que los archivos están:

```
ssh mi-vps
ls ~/proyectos/mi-app-front/web/
# Deberías ver: index.html assets ...
```

7.4 Crear el `docker-compose.yml`

En el servidor, dentro de `~/proyectos/mi-app-front/`:

```
cd ~/proyectos/mi-app-front
nano docker-compose.yml
```

Pega esto (sustituye los valores entre `<...>` por los tuyos):

```
services:
  nginx:
    image: nginx:latest
    expose:
      - "80"
    environment:
      - VIRTUAL_HOST=<midominio.com>
      - LETSENCRYPT_HOST=<midominio.com>
      - LETSENCRYPT_EMAIL=<tu-email@ejemplo.com>
    volumes:
      - ./web:/usr/share/nginx/html:ro
      # Descomenta solo si usas React Router (ver sección 5.6):
      # - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - proxy
    restart: unless-stopped

networks:
  proxy:
    name: nginx-proxy
    external: true
```

7.4.1 Desglose línea a línea

```
image: nginx:latest
```

Usa la imagen oficial de nginx. No necesitamos construir nada custom.

```
expose:  
- "80"
```

Diferencia clave: `expose` hace el puerto disponible solo **dentro de la red Docker**, NO al exterior. Solo `nginx-proxy` puede acceder. **No uses** `ports` aquí: si lo hicieras, el puerto 80 chocaría con el del proxy.

```
environment:  
- VIRTUAL_HOST=midominio.com  
- LETSENCRYPT_HOST=midominio.com  
- LETSENCRYPT_EMAIL=tu-email@ejemplo.com
```

- `VIRTUAL_HOST` : el dominio que `nginx-proxy` asocia a este contenedor. Cuando llega una petición a `midominio.com`, el proxy sabe que va aquí.
- `LETSENCRYPT_HOST` : el dominio para el que `letsencrypt-companion` pedirá certificado HTTPS.
- `LETSENCRYPT_EMAIL` : tu email, para que Let's Encrypt te avise si un certificado va a caducar (en la práctica no recibirás emails porque se renueva solo).

💡 Pueden ser **distintos** entre sí (`VIRTUAL_HOST=midominio.com, www.midominio.com` y `LETSENCRYPT_HOST=midominio.com, www.midominio.com`) si quieres cubrir varios dominios con el mismo contenedor.

```
volumes:  
- ./web:/usr/share/nginx/html:ro
```

Monta tu carpeta `web/` (con los archivos compilados) en la carpeta donde nginx busca los archivos a servir (`/usr/share/nginx/html` por defecto). `:ro` significa "read only", nginx no puede modificar los archivos.

```
networks:  
  - proxy
```

Conecta el contenedor a la red `nginx-proxy`, necesario para que el proxy lo descubra.

```
restart: unless-stopped
```

Si el contenedor se cae o el servidor se reinicia, Docker lo vuelve a arrancar. Excepto si **tú** lo paras a mano.

7.5 Levantar el contenedor

```
docker compose up -d
```

Verifica que está corriendo:

```
docker compose ps
```

Y mira los logs por si hay errores:

```
docker compose logs -f
```

Sale con `Ctrl + C`.

Ahora ve al navegador y abre `http://midominio.com`. La primera vez probablemente:

1. Verás la página unos segundos.
2. Luego se redirigirá a `https://midominio.com` y mostrará el sitio con candadito.

Si el HTTPS no aparece de inmediato, espera 1-2 minutos: el compañero de Let's Encrypt está pidiendo el certificado. Mira los logs para ver el progreso:

```
cd ~/proyectos/nginx-proxy
docker compose logs -f letsencrypt
```

Buscarás algo como `Certificate issued for midominio.com`.

7.6 Si tu app usa React Router (rutas del lado cliente)

Aquí hay una trampa clásica. Si tu app tiene rutas como `/login`, `/dashboard` ... y un usuario:

- Entra por `https://midominio.com` → OK (carga `index.html`, React Router se ocupa del resto).
- Recarga la página estando en `https://midominio.com/dashboard` →  **404**.

¿Por qué? Cuando el navegador pide `/dashboard`, nginx busca un archivo `/usr/share/nginx/html/dashboard` y como no existe, devuelve 404. Pero `/dashboard` es una **ruta de React, no un archivo real**. Necesitamos decirle a nginx que **siempre devuelva** `index.html` (y deje que React Router se ocupe).

7.6.1 Solución

En el servidor, dentro de la carpeta del proyecto:

```
mkdir -p ~/proyectos/mi-app-front/nginx
nano ~/proyectos/mi-app-front/nginx/nginx.conf
```

Pega:

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html =404;
    }
}
```

Lo interesante es `try_files $uri $uri/ /index.html =404`:

- Intenta servir el archivo pedido (`$uri`).

- Si no existe, prueba con una carpeta (`$uri/`).
- Si tampoco, sirve `/index.html` (y React Router se hace cargo de la ruta).
- Si ni siquiera existe `index.html` (no debería pasar), devuelve 404.

Ahora **descomenta** la línea correspondiente en tu `docker-compose.yml` :

```
volumes:
  - ./web:/usr/share/nginx/html:ro
  - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro  # ← esta
```

Y reinicia:

```
docker compose down
docker compose up -d
```

Recarga `https://midominio.com/dashboard` y ahora debería funcionar.

7.7 Actualizar el frontend

Cuando hagas cambios en tu app:

```
# En local:
npm run build
scp -r dist/* mi-vps:~/proyectos/mi-app-front/web/
```

Y ya está. Como nginx sirve los archivos directamente desde la carpeta `web/`, los cambios se ven al instante. No hace falta reiniciar nada.

⚠ Si **no ves los cambios** en el navegador, suele ser caché. Prueba `Ctrl + Shift + R` (recarga forzada) o abre en una ventana de incógnito.

7.8 Resumen visual del flujo

```
[Local: npm run build]
  |
  ▼
[scp dist/* → servidor:~/proyectos/mi-app-front/web/]
```

↓
▼
[docker compose up -d] ← solo la primera vez
↓
▼
[nginx-proxy detecta el VIRTUAL_HOST]
↓
▼
[letsencrypt pide certificado SSL]
↓
▼
[https://midominio.com funcionando ✓]

Frontend desplegado. Si ves el sitio con HTTPS, ¡felicidades! Toca el backend.

8 Desplegar un backend con base de datos

En este capítulo desplegamos un **backend** (Node, Python, lo que sea) junto con una **base de datos** (MySQL en el ejemplo, pero el patrón sirve para PostgreSQL, MongoDB, etc.).

El backend es más complicado que el frontend porque:

- Es código que **se ejecuta** (no son archivos estáticos), así que necesita un proceso vivo.
- Suele necesitar **una base de datos**.
- Maneja **variables de entorno secretas** (contraseñas, claves API...).
- Necesita comunicación **interna y segura** con la base de datos.

Vamos a ir paso a paso.

8.1 Asumimos esta estructura de proyecto

Tu repo del backend debería tener al menos:

```
mi-backend/  
├── Dockerfile           # receta para construir la imagen del backend  
├── docker-compose.yml  # la que vamos a crear ahora  
├── .env.example        # plantilla de variables de entorno  
├── package.json  
├── src/  
└── ...
```

Si no tienes `Dockerfile`, mira el [Anexo: Dockerfile básico](#) al final del capítulo.

8.2 Clonar el repo en el servidor

```
cd ~/proyectos  
git clone git@github.com:tu-usuario/mi-backend.git  
cd mi-backend
```

🔑 Si es un repo privado, necesitas tener configurada la clave SSH del servidor en GitHub (paso 2.6 de la guía).

8.3 Configurar variables de entorno

Las variables de entorno (contraseñas de BBDD, claves de APIs, etc.) **nunca** van en el repositorio. Lo estándar es tener un `.env.example` con la lista de variables (sin valores reales) y crear un `.env` en el servidor con los valores de producción.

```
cp .env.example .env
nano .env
```

Pon valores realistas. Ejemplo:

```
# Servidor
NODE_ENV=production
PORT=3000

# Base de datos
DB_HOST=db
DB_PORT=3306
DB_NAME=miapp
DB_USER=miapp_user
DB_PASSWORD=una-contraseña-larga-y-aleatoria

# JWT u otros secretos
JWT_SECRET=otra-cadena-aleatoria-muy-larga
```

Tres reglas de oro:

1. El archivo `.env` **NUNCA** se sube a Git. Asegúrate de que `.env` está en `.gitignore`.
2. Las contraseñas de producción **no deben coincidir** con las de desarrollo. Genera nuevas (`openssl rand -base64 24` te genera una en el momento).
3. **Fíjate en `DB_HOST=db`**: NO es `localhost` ni una IP. Es `db`, el nombre del servicio en el `docker-compose.yml` que vamos a crear ahora. Más detalles en la sección 6.5.

8.4 Crear el `docker-compose.yml`

```
nano docker-compose.yml
```

Pega esto (sustituye los valores entre `<...>`):

```
services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile
    expose:
      - "3000"
    environment:
      - VIRTUAL_HOST=<api.midominio.com>
      - LETSENCRYPT_HOST=<api.midominio.com>
      - LETSENCRYPT_EMAIL=<tu-email@ejemplo.com>
    env_file:
      - .env
    networks:
      - proxy
      - internal
    restart: unless-stopped
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=<password-root-largo>
      - MYSQL_DATABASE=<miapp>
      - MYSQL_USER=<miapp_user>
      - MYSQL_PASSWORD=<la-misma-que-DB_PASSWORD-del-.env>
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - internal
    restart: unless-stopped

volumes:
  mysql_data:

networks:
  proxy:
    name: nginx-proxy
    external: true
  internal:
    driver: bridge
```

8.4.1 Desglosando este archivo (muy importante para entenderlo)

8.4.1.1 Servicio `backend`

```
build:
  context: .
  dockerfile: Dockerfile
```

A diferencia del frontend que usaba una imagen pública (`nginx:latest`), aquí Docker **construye una imagen** a partir del `Dockerfile` del proyecto. `context: .` significa "el directorio actual".

```
expose:
  - "3000"
```

Otra vez `expose` , no `ports` . El backend solo es accesible para `nginx-proxy` , no directamente desde internet. Cambia 3000 si tu app escucha en otro puerto.

```
env_file:
  - .env
```

Carga las variables del archivo `.env` dentro del contenedor. Mucho más limpio que escribirlas en `environment: .`

```
networks:
  - proxy
  - internal
```

Aquí está la clave de seguridad. El backend está en DOS redes:

- `proxy` → para que `nginx-proxy` pueda mandarle peticiones de internet.
- `internal` → para que pueda hablar con la base de datos.

```
depends_on:
  - db
```

Le dice a Docker que arranque la BBDD **antes** que el backend. Atención: solo espera a que el contenedor `db` arranque, no a que MySQL esté *listo* para aceptar conexiones (eso es típicamente 5-10 segundos más). En la práctica esto basta porque los backends suelen reintentar conectarse.

8.4.1.2 Servicio `db`

```
image: mysql:8.0
```

Usa la imagen oficial de MySQL 8.

```
environment:  
  - MYSQL_ROOT_PASSWORD=...  
  - MYSQL_DATABASE=...  
  - MYSQL_USER=...  
  - MYSQL_PASSWORD=...
```

La imagen oficial de MySQL lee estas variables al arrancar **por primera vez** y:

- Crea la base de datos `MYSQL_DATABASE`.
- Crea el usuario `MYSQL_USER` con la contraseña `MYSQL_PASSWORD`, con permisos sobre esa BBDD.
- Establece la contraseña del usuario `root`.

 Solo se aplica **la primera vez** (mientras el volumen `mysql_data` esté vacío). Si cambias estas variables después, no pasa nada: la BBDD ya está creada. Para recrearla, hay que borrar el volumen (atención, **eso destruye los datos**).

```
volumes:  
  - mysql_data:/var/lib/mysql
```

Lo más importante de toda la configuración de BBDD. MySQL guarda sus datos en `/var/lib/mysql` dentro del contenedor. Si no montamos un volumen, **al reiniciar el contenedor se borra todo**. El volumen `mysql_data` persiste fuera del contenedor.

```
networks:  
  - internal
```

Solo en la red interna. **No** está en `proxy`. Por eso la BBDD nunca es accesible desde internet, aunque alguien sepa la contraseña.

8.4.2 La sección `networks` global

```
networks:
  proxy:
    name: nginx-proxy
    external: true
  internal:
    driver: bridge
```

- `proxy` → la red que ya existe (creada en el capítulo 4), compartida con todos los proyectos. `external: true` significa "usa la que ya hay".
- `internal` → una red **privada para este proyecto**, creada por este compose. Sin `external`. Otros proyectos no la ven.

8.5 ¿Cómo se comunican backend y BBDD?

Aquí pasa la magia de Docker. Dentro de la red `internal`, los servicios se "ven" entre sí por su nombre. Es decir, desde el contenedor `backend` puedes hacer:

```
// Pseudocódigo
connect({ host: 'db', user: 'miapp_user', ... })
```

Docker resuelve `db` automáticamente a la IP del contenedor de la base de datos en la red interna. **No tienes que saber ni configurar ninguna IP.**

Por eso en el `.env` pusimos `DB_HOST=db`. Si lo cambiaras a `localhost` o `127.0.0.1`, el backend buscaría la BBDD **dentro de su propio contenedor**, no la encontraría y daría error.

8.6 Levantar todo

```
docker compose up -d --build
```

El `--build` fuerza a reconstruir la imagen del backend. La primera vez es necesario; en arranques posteriores sin cambios de código puedes omitirlo.

Mira los logs:


```
docker compose logs -f
```

Cosas a vigilar:

- `db_1 | [System] [MS] /usr/sbin/mysqld: ready for connections.` → MySQL está listo.
- Algo tipo `Server running on port 3000` o `Listening on http://0.0.0.0:3000` → tu backend arrancó.
- Errores de conexión a BBDD → revisa que `DB_HOST=db` y las credenciales coinciden con las del servicio `db`.

Comprueba en el navegador:

```
https://api.midominio.com/healthcheck (o la ruta que tengas)
```

Espera 1-2 minutos para que Let's Encrypt genere el certificado.

8.7 Migraciones, seeders, cosas a ejecutar dentro del contenedor

Casi cualquier backend con BBDD necesita ejecutar migraciones la primera vez. Para ejecutar comandos **dentro del contenedor del backend**:

```
docker compose exec backend <comando>
```

Ejemplos:

```
# Migraciones con Sequelize
docker compose exec backend npx sequelize-cli db:migrate

# Migraciones con Prisma
docker compose exec backend npx prisma migrate deploy

# Migraciones con Django
docker compose exec backend python manage.py migrate

# Abrir un shell dentro del contenedor (como SSH)
docker compose exec backend sh
```

8.8 Acceder a la base de datos desde fuera (debug, copias, etc.)

A veces necesitas conectarte a la BBDD con una herramienta (DBeaver, TablePlus, MySQL Workbench...). Como la BBDD **no está expuesta a internet** (a propósito), tienes dos opciones:

8.8.1 Opción A: hacer un túnel SSH (recomendado, seguro)

Desde tu ordenador local:

```
ssh -L 3306:localhost:3306 mi-vps
```

Esto crea un túnel: el puerto 3306 de tu ordenador local se reenvía al puerto 3306 del VPS. Mientras la sesión SSH siga abierta, puedes conectar tu cliente MySQL a `localhost:3306`.



Esto requiere que la BBDD del Docker esté expuesta en el puerto 3306 del host del VPS. Como **no lo está** (es la configuración segura), una versión funcional es entrar al contenedor:

8.8.2 Opción B: entrar al contenedor de BBDD

```
ssh mi-vps
cd ~/proyectos/mi-backend
docker compose exec db mysql -u root -p
# Te pedirá la contraseña MYSQL_ROOT_PASSWORD
```

8.9 Hacer backups (y restaurarlos) de la base de datos

Hacer backups es una de esas cosas que parecen una pérdida de tiempo... hasta el día que las necesitas. **Convierte el backup en un hábito**: por cada cambio importante, por cada despliegue arriesgado, y como mínimo de forma automática cada noche.

Lo importante de un backup no es hacerlo, es **saber restaurarlo**. Haz siempre una prueba de restore al principio del proyecto.

A continuación tienes las recetas para los tres motores más comunes. En todos asumo que el servicio de la base de datos en tu `docker-compose.yml` se llama `db` (cámbialo si no).

8.9.1 MySQL / MariaDB

Crear el backup:

```
cd ~/proyectos/mi-backend  
docker compose exec db mysqldump -u root -p<password> <nombre-bbdd> > backup-$(date +%F).sql
```

Esto te crea un archivo de texto `backup-2026-05-20.sql` con todas las sentencias `CREATE TABLE` e `INSERT` necesarias para recrear la BBDD. Cópialo a tu ordenador:

```
scp mi-vps:~/proyectos/mi-backend/backup-2026-05-20.sql ./
```

Restaurar el backup (sobrescribe la BBDD actual):

```
# Subir el backup al servidor  
scp ./backup-2026-05-20.sql mi-vps:~/proyectos/mi-backend/  
  
# Restaurar  
cd ~/proyectos/mi-backend  
docker compose exec -T db mysql -u root -p<password> <nombre-bbdd> < backup-2026-05-20.sql
```



El `-T` desactiva la asignación de pseudo-terminal: necesario para que la redirección `<` funcione.

Backup de TODAS las bases de datos (útil para migración completa):

```
docker compose exec db mysqldump -u root -p<password> --all-databases > backup-all-$(date +%F).sql
```

8.9.2 PostgreSQL

Crear el backup:

```
cd ~/proyectos/mi-backend
docker compose exec db pg_dump -U <usuario> <nombre-bbdd> > backup-$(date +%F).sql
```

Si tu imagen es la oficial de Postgres y has configurado `POSTGRES_PASSWORD`, te pedirá la contraseña. Para evitarlo en scripts, puedes pasarla con la variable `PGPASSWORD`:

```
docker compose exec -e PGPASSWORD=<password> db pg_dump -U <usuario> <nombre-bbdd> > backup-$(date +%F).sql
```

Formato comprimido (recomendado para BBDD grandes; es binario, no texto):

```
docker compose exec db pg_dump -U <usuario> -F c <nombre-bbdd> > backup-$(date +%F).dump
```

Ocupa bastante menos y permite restaurar tablas selectivamente.

Restaurar un backup en formato SQL (`.sql`):

```
docker compose exec -T db psql -U <usuario> <nombre-bbdd> < backup-2026-05-20.sql
```

Restaurar un backup comprimido (`.dump`):

```
# Copiar el archivo dentro del contenedor primero
docker compose cp backup-2026-05-20.dump db:/tmp/backup.dump

# Restaurar
docker compose exec db pg_restore -U <usuario> -d <nombre-bbdd> --clean -
-if-exists /tmp/backup.dump
```

 `--clean --if-exists` borra primero las tablas existentes antes de restaurar. Si quieres conservar lo que haya y solo añadir, omítelo (puede dar errores de duplicados).

Backup de TODAS las bases de datos del cluster:

```
docker compose exec db pg_dumpall -U <usuario> > backup-all-$(date +%F).sql
```

8.9.3 MongoDB

MongoDB tiene su propio set de herramientas: `mongodump` y `mongorestore`. Crean una carpeta con archivos BSON, no un único `.sql`.

Crear el backup:

```
cd ~/proyectos/mi-backend

# Volcar dentro del contenedor
docker compose exec db mongodump --username <usuario> --password <password> \
  --authenticationDatabase admin \
  --db <nombre-bbdd> \
  --out /tmp/backup

# Copiar fuera del contenedor al host
docker compose cp db:/tmp/backup ./backup-$(date +%F)

# Limpiar dentro del contenedor
docker compose exec db rm -rf /tmp/backup
```

Te queda una carpeta `backup-2026-05-20/` con subcarpetas por cada BBDD. Comprímela para moverla:

```
tar czf backup-2026-05-20.tar.gz backup-2026-05-20/
```

Restaurar el backup:

```
# Descomprimir si es necesario
tar xzf backup-2026-05-20.tar.gz

# Copiar la carpeta dentro del contenedor
docker compose cp ./backup-2026-05-20 db:/tmp/backup

# Restaurar
docker compose exec db mongorestore --username <usuario> --password <password> \
```

```
--authenticationDatabase admin \  
--db <nombre-bbdd> \  
--drop \  
/tmp/backup/<nombre-bbdd>
```

 `--drop` borra las colecciones antes de restaurarlas. Útil para empezar limpio; quítalo si quieres mezclar con datos existentes.

Backup de TODAS las bases de datos:

```
docker compose exec db mongodump --username <usuario> --password <password> \  
--authenticationDatabase admin \  
--out /tmp/backup
```

(Sin `--db`.)

8.9.4 Automatizar backups con `cron`

Para que el backup se haga solo cada noche, en el servidor edita el crontab:

```
crontab -e
```

Y añade una línea (ajusta a tu motor):

```
# Backup MySQL diario a las 3:00  
0 3 * * * cd /home/ubuntu/proyectos/mi-backend && docker compose exec -T  
db mysqldump -u root -ppassword nombre-bbdd > /home/ubuntu/backups/db-$(  
date +%F).sql 2>&1
```

Antes crea la carpeta de destino:

```
mkdir -p ~/backups
```

💡 **No retengas backups eternamente** o te quedas sin disco. Borra los de más de 30 días con otra entrada de cron:

```
0 4 * * * find /home/ubuntu/backups -name "db-*.sql" -mtime +30 -delete
```

8.9.5 Llevarte los backups fuera del servidor (lo importante)

Un backup que vive **solo en el mismo servidor** que la BBDD **no es un backup de verdad**: si el VPS se rompe, pierdes ambos. Lleva los backups a otro sitio:

- Tu ordenador (`scp` semanal).
- Otro VPS distinto.
- Un bucket S3 / Backblaze B2 / similar (búscate la receta del proveedor; es asunto del alumno avanzado).
- Un disco externo donde sea.

Mínimo viable: `scp` **mensual a tu ordenador** del backup más reciente.

8.10 Actualizar el backend

Cuando subas cambios al repo:

```
ssh mi-vps
cd ~/proyectos/mi-backend
git pull
docker compose up -d --build
```

`docker compose` detecta que la imagen cambia (porque el código cambió), la reconstruye, para el contenedor antiguo y arranca el nuevo. **La BBDD sigue intacta** porque sus datos viven en un volumen aparte.

⚠ Si los cambios incluyen nuevas migraciones, ejecútalas después: `docker compose exec backend npx prisma migrate deploy`.

8.11 Anexo: Dockerfile básico para Node.js

Si tu repo no tiene `Dockerfile`, este es un ejemplo razonable para una app Node:

```
FROM node:20-alpine

WORKDIR /app

# Instalar dependencias (las cachea si package.json no cambia)
COPY package*.json ./
RUN npm ci --omit=dev

# Copiar el resto del código
COPY . .

# (Opcional) Compilar si usas TypeScript
# RUN npm run build

EXPOSE 3000

CMD ["node", "src/index.js"]
```

Ajusta el `CMD` según cómo arranque tu app.

¡Y ya está! Si llegaste hasta aquí con todo funcionando, tienes un **stack completo desplegado**: dominio, HTTPS, frontend y backend con base de datos. En el último capítulo recopilamos los problemas comunes que pueden surgir.

Anterior: [← 5. Desplegar frontend](#) | **Siguiente:** [7. Troubleshooting →](#)

9 Troubleshooting: qué hacer cuando algo falla

Este capítulo es el más práctico de toda la guía. Listado de problemas reales que vas a encontrar, ordenados por categoría, con cómo diagnosticarlos y resolverlos.

Antes de buscar tu problema concreto, aprende a leer logs. El 80% de los problemas se resuelve mirando los logs.

9.1 Cómo leer logs (el comando que más vas a usar)

```
cd ~/proyectos/<carpeta-del-proyecto>
docker compose logs -f
```

- `logs` → muestra los logs.
- `-f` → "follow", se queda en vivo mostrando logs nuevos según aparecen.
- `Ctrl + C` → salir (no apaga nada, solo deja de mostrar).

Para un servicio concreto:

```
docker compose logs -f backend
docker compose logs -f db
docker compose logs -f nginx
```

Para los últimos 100 logs (sin seguir en vivo):

```
docker compose logs --tail 100 backend
```



Truco: la primera vez que algo falla, abre dos terminales SSH al servidor. En una pones los logs en vivo, en otra ejecutas comandos. Verás los logs reaccionar a tus pruebas en tiempo real.

9.2 Problemas de conexión SSH

9.2.1 "Permission denied (publickey)"

Significa que la clave no es válida o no la está encontrando.

```
# Comprueba con qué clave intentas conectarte
ssh -v mi-vps
# Lee con atención. Busca líneas como "Trying private key: ..."
```

- Si no usa la clave correcta → revisa tu `~/.ssh/config` que apunte al `IdentityFile` adecuado.
- Si usa la clave pero el servidor la rechaza → la pública no se copió bien al servidor. Repite `ssh-copy-id`.

9.2.2 "Connection refused" o timeout

- El servidor está apagado, o
- Hay un firewall bloqueando el puerto 22, o
- La IP es incorrecta.

Comprueba que el servidor está vivo desde el panel del proveedor. Comprueba que SSH (puerto 22) está abierto en el firewall.

9.2.3 Se queda colgado al conectar

A veces es DNS lento o problemas de red. Prueba con la IP directamente en vez del nombre:

```
ssh ubuntu@123.45.67.89
```

9.3 Problemas con Docker

9.3.1 "docker: command not found"

No has instalado Docker. Vuelve al capítulo 3.

9.3.2 "Got permission denied while trying to connect to the Docker daemon socket"

No estás en el grupo `docker`. Repite:

```
sudo usermod -aG docker $USER
newgrp docker
# 0 simplemente cierra sesión y vuelve a conectarte
```

9.3.3 "Cannot connect to the Docker daemon"

Docker no está arrancado:

```
sudo systemctl status docker
sudo systemctl start docker
sudo systemctl enable docker # para que arranque siempre con el sistema
```

9.3.4 "Error response from daemon: ... port is already allocated"

Otro contenedor ya está usando ese puerto. Mira quién:

```
docker ps
# Verás los contenedores y los puertos que ocupan
```

Para liberar:

```
docker stop <nombre-o-id>
# o si es el que ya no quieres
docker rm -f <nombre-o-id>
```

9.3.5 El contenedor arranca y se cierra inmediatamente

Mira los logs justo después:

```
docker compose logs backend
```

Suele ser un error en el código (excepción al arrancar), una variable de entorno faltante, o un fallo de conexión a la BBDD.

9.4 Problemas con `nginx-proxy`

9.4.1 Veo "503 Service Temporarily Unavailable" al entrar al dominio

Significa que `nginx-proxy` recibió la petición pero no encontró ningún contenedor que sirva ese dominio. Posibles causas:

1. El contenedor de tu proyecto no está arrancado:

```
cd ~/proyectos/mi-app-front
docker compose ps
# ¿Aparece como "Up"?
```

2. El contenedor de tu proyecto no está en la red `nginx-proxy` :

```
docker network inspect nginx-proxy
# Mira "Containers": debe estar el tuyo
```

Si no está, revisa que tu `docker-compose.yml` tenga `networks: - proxy` y la sección global `networks: proxy: name: nginx-proxy external: true`.

3. `VIRTUAL_HOST` no coincide con el dominio que pides: Revisa que el valor de `VIRTUAL_HOST` en tu compose sea **exactamente** el dominio que pruebas (sin `https://`, sin `www.` si no lo configuraste).

9.4.2 El certificado SSL no se genera

Mira los logs del compañero:

```
cd ~/proyectos/nginx-proxy
docker compose logs --tail 100 letsencrypt
```

Causas comunes:

- **DNS no propagado:** Let's Encrypt necesita verificar que controlas el dominio. Si el dominio aún no apunta al servidor, falla. Espera y comprueba con `dig +short midominio.com`.
- **Puerto 80 cerrado:** Let's Encrypt hace la verificación por HTTP. Si el puerto 80 está bloqueado, no puede verificar.

- **Has pedido el certificado demasiadas veces** (rate limit de Let's Encrypt). Tienes un límite de unos 5 certificados por dominio por semana. Espera 7 días o usa el endpoint staging para pruebas.
- **El email es inválido.**

9.4.3 "Too many redirects" o el sitio entra en bucle

Casi siempre tu backend está intentando redirigir HTTP→HTTPS por su cuenta, pero como detrás del proxy todo el tráfico le llega como HTTP, el bucle es infinito. La solución es que tu backend **no haga la redirección**, deja que `nginx-proxy` la haga (lo hace por defecto). Si tu framework tiene un middleware tipo "force HTTPS", desactívalo.

9.5 Problemas con el frontend

9.5.1 Página en blanco

1. Abre las **DevTools del navegador** (`F12`) y mira la pestaña *Console*. ¿Errores rojos?
2. Mira la pestaña *Network*. ¿Los archivos JS y CSS cargan con código 200?
3. ¿Tu `index.html` está realmente en `~/proyectos/mi-app-front/web/` ?

```
ls ~/proyectos/mi-app-front/web/  
# Debe haber al menos index.html
```

9.5.2 Funciona la home pero al refrescar una ruta interna da 404

Es React Router. Te falta el `nginx.conf` con `try_files` . Mira la sección 5.6 del capítulo 5.

9.5.3 Las llamadas a la API fallan con CORS

Si frontend (`midominio.com`) y backend (`api.midominio.com`) están en dominios distintos, el navegador pide al backend que mande cabeceras CORS. En tu backend tienes que permitir el origen:

```
// Ejemplo Express  
import cors from 'cors';  
app.use(cors({ origin: 'https://midominio.com' })));
```

9.5.4 Los cambios del frontend no aparecen

Caché del navegador. Prueba `Ctrl + Shift + R` (recarga forzada) o abre ventana de incógnito. Si persiste, verifica que la subida con `scp` se hizo realmente:

```
ls -la ~/proyectos/mi-app-front/web/  
# Mira la fecha de modificación de index.html
```

9.6 Problemas con el backend

9.6.1 El backend no se conecta a la base de datos

Lo más probable es uno de estos:

1. **DB_HOST mal configurado en .env**: debe ser `db` (el nombre del servicio en compose), NO `localhost`, NO `127.0.0.1`, NO la IP del servidor.
2. **Las credenciales no coinciden**: las variables `MYSQL_USER` / `MYSQL_PASSWORD` del servicio `db` deben coincidir con `DB_USER` / `DB_PASSWORD` del `.env` que lee el backend.
3. **Servicios en redes distintas**: ambos deben estar en la red `internal`.
4. **MySQL aún no está listo**: a veces el backend arranca antes de que MySQL esté listo, falla, y como tienes `restart: unless-stopped` Docker lo reinicia hasta que sí lo encuentra. Mira los logs, si en 30 segundos termina arrancando bien, no es un problema. Si no, hay otra causa.

9.6.2 "Connection refused" al hacer peticiones a la API

Pasos a comprobar:

```
# 1. ¿El contenedor está corriendo?  
docker compose ps  
  
# 2. ¿En el log dice "listening on..." en el puerto correcto?  
docker compose logs backend | tail -20  
  
# 3. ¿La app escucha en 0.0.0.0 (todas las interfaces) y no solo en localhost?  
# Esto es un error MUY común. Tu app debe arrancar así:  
#   app.listen(3000, '0.0.0.0')  
# NO así:  
#   app.listen(3000, 'localhost')  
# Porque "localhost" dentro del contenedor no es accesible desde fuera del contenedor.
```

9.6.3 Las migraciones no se ejecutan automáticamente

Tienes que ejecutarlas a mano la primera vez:

```
docker compose exec backend <comando-de-migrar>
```

Algunas opciones avanzadas: añadir un script de entrada (`entrypoint.sh`) que las ejecute antes de arrancar el server. Pero para empezar, ejecútalas a mano.

9.6.4 Los datos de la BBDD desaparecen al reiniciar

Revisa que tienes el volumen:

```
db:
  ...
  volumes:
    - mysql_data:/var/lib/mysql  # ← ESTO
  ...

volumes:
  mysql_data:                    # ← Y ESTO
```

Si te falta una de las dos partes, los datos viven dentro del contenedor y se borran al recrearlo.

9.7 Problemas de DNS y dominio

9.7.1 "No se puede acceder a este sitio"

Empieza por lo básico:

```
# ¿El dominio resuelve?
dig +short midominio.com

# ¿Devuelve la IP de tu VPS?
```

Si **no devuelve nada o devuelve una IP incorrecta**: revisa los registros DNS en tu registrador. Recuerda que los cambios tardan en propagarse.

Si **devuelve la IP correcta pero no puedes conectar**: el problema está en el servidor, no en DNS.

9.7.2 El subdominio `www.` no funciona

Por defecto, configurar `midominio.com` no incluye `www.midominio.com`. Tienes que añadir el registro DNS para `www` y declararlo en `VIRTUAL_HOST`:

```
- VIRTUAL_HOST=midominio.com,www.midominio.com
- LETSENCRYPT_HOST=midominio.com,www.midominio.com
```

Y reiniciar el contenedor.

9.8 Comandos de diagnóstico que conviene memorizar

Comando	Para qué sirve
<code>docker ps</code>	Ver qué contenedores están corriendo.
<code>docker ps -a</code>	Ver también los parados (con su código de salida).
<code>docker compose logs -f <servicio></code>	Logs en vivo de un servicio.
<code>docker network ls</code>	Ver las redes Docker existentes.
<code>docker network inspect <red></code>	Ver qué contenedores hay en una red.
<code>docker compose exec <serv> sh</code>	Abrir un shell dentro de un contenedor.
<code>docker stats</code>	Uso de CPU/RAM en vivo de los contenedores.
<code>df -h</code>	Espacio libre en disco del servidor.
<code>free -h</code>	RAM disponible.
<code>dig +short <dominio></code>	A qué IP resuelve un dominio.
<code>curl -I https://<dominio></code>	Ver cabeceras HTTP de una URL (status, redirects...).
<code>sudo ufw status</code>	Estado del firewall.

9.9 Reiniciar todo "porque sí" cuando estás bloqueado

A veces lo más rápido es:


```
# El proxy (NO suele necesitar reinicios, pero por si acaso)
cd ~/proyectos/nginx-proxy
docker compose down
docker compose up -d

# Tu proyecto
cd ~/proyectos/mi-proyecto
docker compose down
docker compose up -d --build
```

Esto NO borra los volúmenes (los datos persisten). Para borrar también los volúmenes (⚠ pierdes datos):

```
docker compose down -v
```

9.10 Borrar cosas selectivamente sin cargarte la base de datos

Este es uno de los puntos donde más fácil es meter la pata. Vamos a verlo con detalle, porque mezclar mal `down`, `down -v`, `prune` o borrar volúmenes a mano puede destruirte la BBDD de producción en un segundo.

9.10.1 Entender qué es cada cosa

Antes de borrar nada, ten claro qué tipos de "objetos" hay en Docker:

Objeto	Qué es	¿Borrarlo pierde datos?
Contenedor	Una instancia en ejecución. Efímera por naturaleza.	No (los datos viven fuera).
Imagen	La "plantilla" desde la que se crean contenedores.	No.
Volumen	Carpeta persistente. Aquí viven los datos de la BBDD, uploads, etc.	Sí. Cuidado.
Red	El "cable" virtual que conecta contenedores entre sí.	No, pero rompe conexiones.
Build cache	Capas intermedias que Docker reutiliza al construir imágenes.	No, solo enlentece la próxima build.

Regla de oro: los volúmenes son sagrados, son donde viven tus datos. Todo lo demás es desechable.

9.10.2 Ver qué volúmenes tienes

```
docker volume ls
```

Verás algo como:

DRIVER	VOLUME NAME	
local	mi-backend_mysql_data	← LA BASE DE DATOS, NO TOCAR
local	nginx-proxy_certs	← certificados HTTPS, no tocar
local	nginx-proxy_vhost	
local	ed8f3c2a9b...	← volumen anónimo huérfano, suele ser seguro borrar

Los volúmenes con nombre legible (`<proyecto>_<nombre>`) son los que tú declaraste en tus `docker-compose.yml` . Los que son solo un hash largo son **volúmenes anónimos** creados por imágenes que declaran `VOLUME` en su Dockerfile (por ejemplo, `node:alpine` lo hace para `node_modules` en algunos casos), y suelen ser huérfanos cuando ya nadie los usa.

Para ver dónde está físicamente un volumen:

```
docker volume inspect mi-backend_mysql_data
# Te muestra "Mountpoint": dónde lo guarda Docker en el host
(típicamente /var/lib/docker/volumes/...)
```

9.10.3 Caso 1: reconstruir el backend porque cambié código

El más habitual. Quieres reconstruir la imagen del backend para que coja los cambios de `git pull` , pero **NO** quieres tocar la base de datos.

```
cd ~/proyectos/mi-backend
git pull
docker compose up -d --build
```

`docker compose up -d --build` reconstruye solo las imágenes que has marcado para build y recrea los contenedores. El volumen `mysql_data` ni se toca: el contenedor `db` se reinicia (o ni eso, si su imagen no cambió) y vuelve a montar el mismo volumen.

Si quieres ser explícito y reconstruir solo el backend, sin tocar el `db`:

```
docker compose up -d --build backend
```

9.10.4 Caso 2: forzar reinstalación de `node_modules` (o equivalente)

A veces tras un `git pull` el contenedor del backend arranca pero falla porque el `node_modules` cacheado dentro de la imagen está desactualizado. Lo que quieres es: **borrar la imagen del backend y reconstruirla desde cero**, sin tocar nada más.

```
cd ~/proyectos/mi-backend

# Paramos solo el backend, no el db
docker compose stop backend
docker compose rm -f backend

# Borramos la imagen del backend para forzar rebuild completo
docker compose build --no-cache backend

# Volvemos a arrancar (db ni se entera)
docker compose up -d
```

`--no-cache` ignora el cache de capas de Docker y reinstala todo desde cero (incluido `npm ci`). Tarda más pero garantiza que no quedan restos.



Cuándo usar `--no-cache`: cuando tras un `git pull` con cambios en `package.json` la app sigue rota. Cuando cambias variables que afectan a la build. Cuando "no entiendo por qué no coge mis cambios".

9.10.5 Caso 3: tienes un volumen anónimo para `node_modules` que quieres limpiar

Algunos `docker-compose.yml` montan `node_modules` como volumen anónimo para evitar conflictos entre el `node_modules` del host y el del contenedor. Algo como:

```
backend:
  volumes:
```

- ./app
- /app/node_modules # ← volumen anónimo

Cuando cambias dependencias y quieres que se reinstalen limpias, tienes que borrar **ese volumen específico**, sin tocar `mysql_data`.

```
# 1. Ver el ID del volumen anónimo que pertenece al contenedor del backend
docker inspect mi-backend-backend-1 | grep -A 5 Mounts
# Busca el "Source" del volumen montado en /app/node_modules

# 2. Parar y borrar el contenedor del backend (NO del db)
docker compose stop backend
docker compose rm -f -v backend
# El -v borra los volúmenes ANÓNIMOS asociados a ese contenedor

# 3. Reconstruir
docker compose up -d --build backend
```

Importante: `docker compose rm -f -v` solo borra los volúmenes anónimos del contenedor que estás eliminando. Los volúmenes **con nombre** declarados en la sección global `volumes:` (como `mysql_data`) NO se borran con esto. Eso es exactamente lo que queremos.

9.10.6 Caso 4: quiero resetear COMPLETAMENTE el backend, pero conservar la BBDD

```
cd ~/proyectos/mi-backend

# Paramos todos los servicios del proyecto
docker compose stop

# Borramos contenedores y volúmenes anónimos, pero NO los volúmenes con nombre
docker compose rm -f -v

# Forzamos rebuild de las imágenes desde cero
docker compose build --no-cache

# Arrancamos todo de nuevo
docker compose up -d
```

El volumen `mysql_data` sobrevive porque está declarado con nombre en `volumes:` y `compose rm` no toca los volúmenes con nombre.

9.10.7 Caso 5: quiero borrar TAMBIÉN la base de datos (resetear todo)

Solo cuando de verdad quieras empezar de cero (por ejemplo, en un entorno de desarrollo, nunca en producción sin un backup):

```
cd ~/proyectos/mi-backend
docker compose down -v
```

`down -v` para los contenedores, los borra, y **borra los volúmenes con nombre** asociados al compose. Tu BBDD queda como en el día 1.

⚠ Antes de hacer esto en cualquier entorno con datos importantes, **haz backup primero** (ver sección 6.9 del capítulo de backend).

9.10.8 Caso 6: borrar un volumen específico a mano

Si por lo que sea quieres borrar un volumen concreto:

```
# Primero asegúrate de que ningún contenedor lo está usando
docker volume inspect mi-backend_mysql_data
# Si "RefCount" o el contenedor que lo usa aparece, páralo antes

# Borra el volumen
docker volume rm mi-backend_mysql_data
```

Si te da error "volume is in use", significa que algún contenedor todavía lo tiene montado. Páralo primero con `docker compose stop` y reintentá.

9.10.9 Tabla resumen: ¿qué borra cada comando?

Comando	Contenedores	Imágenes	Volúmenes anónimos	Volúmenes con nombre
<code>docker compose stop</code>	⏸	✗	✗	✗
<code>docker compose down</code>	✓	✗	✗	✗
<code>docker compose down -v</code>	✓	✗	✓	✓

Comando	Contenedores	Imágenes	Volúmenes anónimos	Volúmenes con nombre
<code>docker compose rm -f</code>	✓	✗	✗	✗
<code>docker compose rm -f -v</code>	✓	✗	✓	✗
<code>docker compose build --no-cache</code>	✗	↺	✗	✗

Leyenda: ✓ borra, ✗ no toca, ⏸ para sin borrar, ↺ reconstruye.

Memoriza esto: mientras no escribas `down -v` o `volume rm` con un nombre explícito, **tu base de datos está a salvo**.

9.11 Quedarse sin espacio en disco

Docker, con el tiempo, acumula imágenes viejas, capas intermedias y volúmenes huérfanos. Revisa:

```
df -h          # cuánto espacio libre queda en /
docker system df # cuánto espacio ocupa Docker
```

Limpieza segura:

```
docker system prune # borra contenedores parados, redes sin uso
e imágenes colgadas
docker system prune -a # MÁS AGRESIVO: borra también imágenes que
ninguna otra está usando
docker volume prune # borra volúmenes huérfanos (sin contenedor
asociado)
```

⚠ `docker volume prune` puede borrar el volumen de una BBDD si su contenedor está parado. Ten cuidado.

9.12 Cuando pedir ayuda

Si después de leer este capítulo sigues bloqueado, antes de pedir ayuda, **prepara esta información**:

1. ¿Qué intentas hacer?

2. ¿Qué pasa exactamente? (mensaje de error literal)
3. Salida de `docker compose ps`.
4. Últimas 50 líneas de `docker compose logs --tail 50` del servicio que falla.
5. Tu `docker-compose.yml` (sin contraseñas).
6. Pasos que ya has probado.

Esto facilita muchísimo que alguien (humano o IA) pueda ayudarte. La mayoría de "no me funciona" se resuelve solo con leer los logs detenidamente.

Anterior: [← 6. Desplegar backend](#) | **Siguiente:** [8. Migrar a otro VPS →](#)

10 Migrar a un VPS nuevo

Llega un día en el que necesitas **mover todo a otro servidor**: porque el actual sale caro, porque has encontrado un proveedor mejor, porque te has quedado corto de RAM, o porque cambias de país y quieres datacenter más cerca.

Este capítulo es una guía paso a paso para migrar **todos tus proyectos** de un VPS a otro:

- Sin perder datos (ni de BBDD, ni de archivos subidos por usuarios).
- Con el menor downtime posible (idealmente menos de 5 minutos).
- Sin romper los HTTPS (los certificados se regeneran solos).



Lo bonito de tener todo dockerizado es esto: **migrar es básicamente "tar + scp + docker compose up"**. Sin Docker, esta misma migración te llevaría días.

10.1 Conceptos previos: ¿qué hay que mover?

Antes de empezar, ten claro qué cosas tienes en el VPS viejo que necesitas en el nuevo. Hay **cuatro tipos**:

1. **Código de los proyectos** (`docker-compose.yml` , `Dockerfile` , archivos compilados del frontend...). Está en `~/proyectos/` . Algunos están en repos de Git (puedes hacer `git clone` en el nuevo servidor), otros no (frontends compilados, configuración custom).
2. **Datos de las bases de datos**. Viven en volúmenes de Docker. Esto es **lo crítico**: lo único que **no se puede regenerar** si lo pierdes.
3. **Archivos subidos por usuarios** si los hay (avatares, documentos, etc.). Pueden vivir en otro volumen, o en una carpeta montada del host.
4. **Configuración del sistema**: claves SSH, configuración de firewall, crontab... Esto se reconfigura a mano, no se "migra".

Lo que **NO** necesitas mover:

- Los **certificados HTTPS** (se regeneran solos en el VPS nuevo).
- Las **imágenes Docker** (se reconstruyen / se descargan solas).
- Los **contenedores** (se recrean en el nuevo).

10.2 Estrategia general

El plan es:

1. **Preparar el VPS nuevo**: capítulos 2, 3 y 4 de esta guía (SSH, Docker, nginx-proxy).

2. **Hacer un dump completo de cada BBDD** en el viejo.
3. **Copiar todo al VPS nuevo:** código + dumps + archivos de usuario.
4. **Restaurar y levantar todo en el nuevo**, probando sin tocar DNS aún.
5. **Cambiar el DNS** para que apunte al nuevo.
6. **Esperar a comprobar que todo va bien** y, días después, dar de baja el viejo.

El downtime real es solo el tiempo entre el último dump y el momento en que el DNS apunta al nuevo. Si lo haces a las 4 AM, en la práctica es invisible.

10.3 Antes de empezar: una checklist

- [] ¿Tengo acceso SSH al servidor viejo? Pruébalo: `ssh mi-vps-viejo`.
- [] ¿Tengo el VPS nuevo contratado y la IP apuntada?
- [] ¿Sé qué proyectos están en el servidor viejo y qué BBDD usa cada uno? Si no, conéctate y revisa:

```
ls ~/proyectos
cd ~/proyectos/cada-uno && cat docker-compose.yml | grep -A 2 "image:"
```

- [] ¿Sé qué dominios apuntan al servidor viejo? Lista los DNS de tu registrador.
- [] ¿Tengo a mano las **credenciales de cada** `.env`? Las vas a necesitar para que el VPS nuevo conecte a las mismas BBDD con los mismos usuarios. Si están solo en el servidor, no las pierdas en el camino.

 Lee la guía entera antes de empezar. Hay un par de detalles en el paso del DNS que conviene tener claros antes de actuar.

10.4 Paso 1: preparar el VPS nuevo

En el VPS nuevo, ejecuta de principio a fin:

1. Capítulo 2: [conexión SSH y claves](#) — configura tu acceso, añade alias `mi-vps-nuevo` en `~/.ssh/config`.
2. Capítulo 3: [instalar Docker](#) — Docker + permisos.
3. Capítulo 4: [nginx-proxy](#) — proxy reverso + Let's Encrypt. **NO toques aún el DNS de los dominios.**

Cuando termines, el VPS nuevo debería tener:

- Docker funcionando.
- La red `nginx-proxy` creada.

- Los contenedores `nginx-proxy` y `letsencrypt` corriendo.
- Pero **sin ningún proyecto desplegado**.

💡 Esto lo puedes hacer **días antes** del corte real. Cuanto antes esté preparado, más tranquila la migración.

10.5 Paso 2: dumps de las bases de datos en el viejo

Conéctate al VPS viejo (`ssh mi-vps-viejo`) y haz dump de cada BBDD.

Si solo tienes un proyecto con BBDD:

```
cd ~/proyectos/mi-backend
mkdir -p ~/migration

# Ajusta el comando según tu motor (ver capítulo 6.9):
docker compose exec -T db mysqldump -u root -p<password> --all-databases
> ~/migration/mysql-dump.sql
# o
docker compose exec -T db pg_dumpall -U <usuario> > ~/migration/postgres-
dump.sql
# o
docker compose exec db mongodump --username <usuario> --password <passwo
rd> --authenticationDatabase admin --out /tmp/mongo
docker compose cp db:/tmp/mongo ~/migration/mongo
```

Si tienes varios proyectos con BBDD diferentes, repite por cada uno. Pon nombres claros (`mi-backend-mysql.sql` , `blog-postgres.sql` , etc.).

🔑 **Importante:** hazlo **antes** de la ventana de corte para verificar que funciona. Luego, justo antes de migrar, **vuelves a hacerlo** para tener el dump más reciente.

10.6 Paso 3: empaquetar todo lo que hay que mover

Sigue en el VPS viejo. Vamos a juntar en una carpeta `migration/` todo lo que necesitas:

```
cd ~/migration
mkdir -p proyectos

# Por cada proyecto, copia su carpeta (excluye node_modules, cosas
# pesadas que se regeneran)
for proj in ~/proyectos/*; do
```

```

name=$(basename "$proj")
if [ "$name" != "nginx-proxy" ]; then
# el proxy ya está montado en el nuevo
    rsync -a --exclude='node_modules' --exclude='.next' --exclude='dist'
--exclude='build' "$proj" "./proyectos/$name/"
    fi
done

# Si tu frontend tiene archivos compilados que NO están en Git,
inclúyelos:
# (en este ejemplo el frontend vive en mi-app-front/web/)
rsync -a ~/proyectos/mi-app-front/web/ ./proyectos/mi-app-front/web/

```

Esto te crea `~/migration/proyectos/` con todo el código.

Si tienes archivos de usuario (uploads), no te los olvides. Si están en un volumen Docker con nombre, mira la sección 8.10 más abajo. Si están en una carpeta del host (típico patrón `./uploads:/app/uploads` en el compose), entonces ya los has copiado con el `rsync` de arriba.

Empaqueta todo en un tarball:

```

cd ~
tar czf migration.tar.gz migration/
ls -lh migration.tar.gz

```

10.7 Paso 4: transferir al VPS nuevo

Tienes dos opciones, según donde te resulte más cómodo.

Opción A: directa VPS viejo → VPS nuevo (más rápido, sin pasar por tu ordenador). Desde el VPS viejo:

```

# Necesitas haber añadido tu clave SSH del viejo a las authorized_keys
del nuevo
scp ~/migration.tar.gz mi-vps-nuevo:~/

```

Si no tienes SSH entre servidores, generamos una clave en el viejo y la añadimos al nuevo:

```

# En el VPS viejo
ssh-keygen -t ed25519 -C "migration" # acepta defaults
cat ~/.ssh/id_ed25519.pub
# Copia la salida

```

```
# En el VPS nuevo
echo "PEGA_AQUI_LA_CLAVE" >> ~/.ssh/authorized_keys
```

Y ahora `scp ~/migration.tar.gz <user>@<ip-nuevo>:~/` funciona.

Opción B: pasar por tu ordenador (más lento, pero útil si quieres tener el backup también en local):

```
# En tu ordenador
scp mi-vps-viejo:~/migration.tar.gz ./
scp ./migration.tar.gz mi-vps-nuevo:~/
```

10.8 Paso 5: levantar todo en el VPS nuevo

Conéctate al VPS nuevo y descomprime:

```
ssh mi-vps-nuevo
cd ~
tar xzf migration.tar.gz
mv migration/proyectos/* ~/proyectos/
ls ~/proyectos/
# Deberías ver tus proyectos + nginx-proxy
```

Restaura las BBDD ANTES de levantar los backends (si no, el backend intentará conectarse a una BBDD vacía y fallará).

Por cada proyecto con BBDD:

```
cd ~/proyectos/mi-backend

# Arranca SOLO la base de datos
docker compose up -d db

# Espera 10-15 segundos a que esté lista (mira los logs si dudas)
docker compose logs db | tail

# Restaura el dump (ajusta según motor; ver capítulo 6.9)
docker compose exec -T db mysql -u root -p<password> < ~/migration/mysql-
dump.sql
# o
docker compose exec -T db psql -U <usuario> < ~/migration/postgres-
dump.sql
```

```
# o (Mongo)
docker compose cp ~/migration/mongo db:/tmp/mongo
docker compose exec db mongorestore --username <user> --password <pwd> --
authenticationDatabase admin /tmp/mongo

# Una vez restaurado, arranca el resto
docker compose up -d
```

Para proyectos sin BBDD (frontends estáticos):

```
cd ~/proyectos/mi-app-front
docker compose up -d
```

10.9 Paso 6: probar antes de tocar DNS

Aquí está el truco que evita el "y si no funciona, los usuarios están sin servicio": **probar el VPS nuevo SIN cambiar el DNS público.**

En **tu ordenador**, edita el archivo de hosts para que tu navegador, **solo para ti**, resuelva `midominio.com` a la nueva IP.

En Linux/macOS:

```
sudo nano /etc/hosts
```

En Windows:

```
C:\Windows\System32\drivers\etc\hosts    (como administrador)
```

Añade líneas como:

```
123.45.67.89 midominio.com
123.45.67.89 api.midominio.com
123.45.67.89 www.midominio.com
```

Donde `123.45.67.89` es la IP **del VPS nuevo**.

Ahora abre `https://midominio.com` en tu navegador.

⚠ La primera vez probablemente verás un error de certificado: Let's Encrypt aún no ha emitido el certificado del nuevo. Espera 1-2 minutos y recarga. Si tras 5 minutos sigue fallando, mira los logs:

```
docker compose -f ~/proyectos/nginx-proxy/docker-compose.yml logs 1
etsencrypt | tail -50
```

(Pista: para que Let's Encrypt funcione, el dominio debe resolver públicamente al VPS nuevo. Como aún no has cambiado el DNS, no resuelve. **Para hacer la prueba COMPLETA del HTTPS necesitas cambiar el DNS primero.** Si solo quieres probar HTTP sin certificado, entra a `http://midominio.com` aceptando el aviso.)

Comprobaciones que conviene hacer antes del cambio de DNS:

- [] El frontend carga (al menos en HTTP).
- [] El backend responde (`curl http://api.midominio.com/healthcheck` desde el VPS nuevo).
- [] Conexión backend ↔ BBDD funciona: prueba un endpoint que lea datos. Verifica que **los datos del dump están** (entra como un usuario que existía, mira si está su perfil).
- [] Las migraciones están aplicadas (si tu framework lo gestiona):

```
docker compose exec backend npx prisma migrate status # o el
equivalente
```

No te olvides de quitar las líneas del `/etc/hosts` cuando termines la prueba, o tu ordenador siempre irá al nuevo aunque el DNS apunte al viejo y te volverás loco depurando.

10.10 Paso 7 (opcional): copiar volúmenes Docker enteros

Si tienes datos en volúmenes Docker con nombre que **no son una BBDD** (uploads, archivos generados, etc.) y no se pueden dumppear con un comando como `mysqldump`, puedes copiar el volumen entero.

En el VPS viejo, exporta el volumen a un tarball:

```
docker run --rm \
-v mi-backend_uploads:/data \
-v ~/migration:/backup \
```

```
alpine \
tar czf /backup/uploads.tar.gz -C /data .
```

Desglose:

- Crea un contenedor temporal Alpine.
- Monta el volumen `mi-backend_uploads` en `/data`.
- Monta tu carpeta `~/migration` en `/backup`.
- Hace tar de todo el contenido del volumen.
- Se autoborra al terminar (`--rm`).

Copia `uploads.tar.gz` al VPS nuevo (junto con los demás archivos del paso 8.7).

En el VPS nuevo, primero levanta el proyecto al menos una vez (`docker compose up -d`) para que se cree el volumen vacío, luego rellénalo:

```
docker compose stop # para que ningún contenedor esté usando el volumen
docker run --rm \
-v mi-backend_uploads:/data \
-v ~/migration:/backup \
alpine \
sh -c "cd /data && tar xzf /backup/uploads.tar.gz"

docker compose up -d
```

💡 Truco mental: con este patrón puedes **mover cualquier volumen Docker** entre máquinas. Es la forma genérica de migrar lo que `mysqldump` y compañía no cubren.

10.11 Paso 8: el cambio de DNS (el momento de la verdad)

Cuando hayas verificado que el VPS nuevo funciona, llega el corte real. Hazlo en una franja de poca actividad de tus usuarios.

10.11.1 Antes del corte: bajar el TTL del DNS

El **TTL** (Time To Live) de un registro DNS es cuánto tiempo lo cachean los resolvers. Si tu TTL es de 24 horas, después de cambiar la IP algunos usuarios seguirán yendo al viejo **durante un día**.

24-48 horas ANTES de migrar, baja el TTL a 300 segundos (5 minutos) desde el panel de tu registrador. Así, cuando hagas el cambio, la propagación es rápida.

10.11.2 El corte: secuencia paso a paso

1. **En el VPS viejo: hacer un último dump** (el más reciente posible). Repite el paso 8.5.
2. **Pasar el dump al nuevo** y restaurarlo encima del que ya restauraste antes:

```
# En el viejo
scp ~/migration/mysql-dump.sql mi-vps-nuevo:~/migration/mysql-
dump-final.sql

# En el nuevo
cd ~/proyectos/mi-backend
docker compose exec -T db mysql -u root -p<password> < ~/
migration/mysql-dump-final.sql
```

3. **Poner el VPS viejo en "modo solo lectura"** (opcional pero recomendado): apaga los backends para que nadie pueda escribir más datos que se perderían:

```
# En el viejo
cd ~/proyectos/mi-backend
docker compose stop backend
```

4. **Cambiar los registros DNS** desde el panel del registrador: pon la nueva IP en todos los registros `A`.
5. **Esperar a que el DNS propague**. Como bajaste el TTL antes, suele tardar 5-15 minutos. Comprueba:

```
dig +short midominio.com
# Cuando devuelva la IP del nuevo, ha propagado
```

6. **Volver a entrar a tu sitio** desde el navegador (sin las líneas de `/etc/hosts`) y comprobar todo.

10.11.3 Tras el corte

- **El primer HTTPS puede tardar 1-2 minutos en aparecer** mientras Let's Encrypt emite el certificado. Si tarda más, revisa los logs (puede ser que el DNS no haya propagado del todo).
- **Vuelve a subir el TTL** del DNS a su valor normal (3600 o 86400 segundos) si lo bajaste.

- **Mantén el VPS viejo encendido al menos 1-2 semanas más**, por si te encuentras un problema y necesitas dumppear algo otra vez, o por si descubres un usuario despistado cuyo DNS aún no ha actualizado.

10.12 Paso 9: dar de baja el VPS viejo

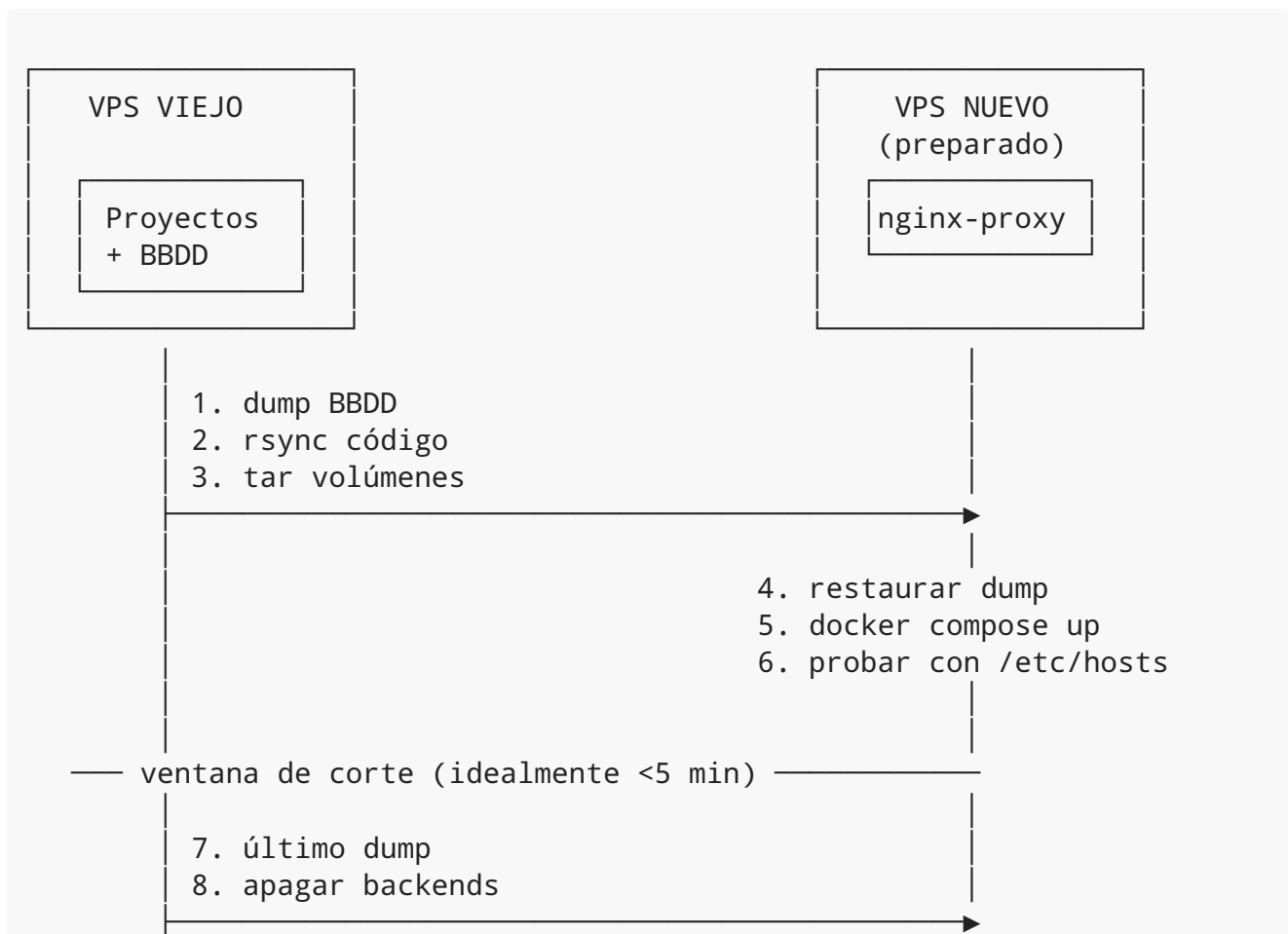
Antes de cancelar el VPS viejo:

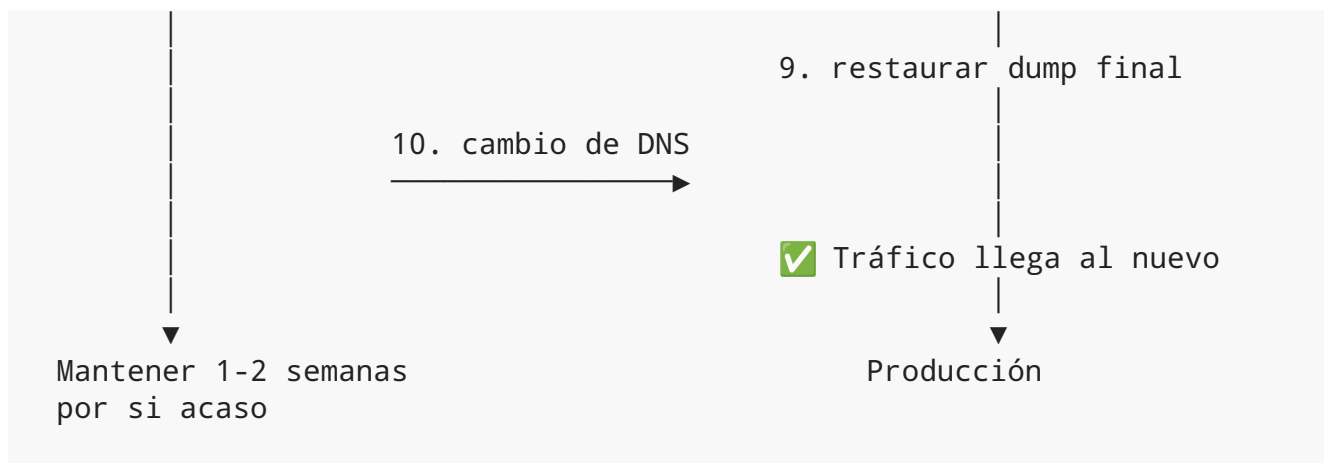
- [] Llevas al menos 1 semana funcionando en el nuevo sin incidentes.
- [] Has hecho un backup completo de las BBDD del viejo y lo tienes en otro sitio (ordenador local, S3...).
- [] Has revisado que no quedan dominios apuntando al viejo en tu panel DNS.
- [] Has revisado que ningún usuario reciente reporta problemas.

Cuando estés tranquilo, cancela el VPS viejo desde el panel de tu proveedor.

💡 Si tu proveedor te permite hacer una **snapshot** del VPS antes de borrarlo, hazla. Te cuesta unos céntimos y te da una red de seguridad por si algo asoma semanas después.

10.13 Resumen visual del proceso





Con esto cierras el ciclo completo: alquilar un VPS, montar Docker, desplegar proyectos, gestionar problemas, hacer backups y migrar a un servidor nuevo cuando convenga.